

BDH Driven Continuous Narrative Reasoning

Team BDH_ON_TOP

Piyush Panda, Priyam Ghosh, Priyansh Rawat

January 2026

Index

Contents

1	Introduction	3
2	Project Framing and Whiteboard Walkthrough	3
2.1	What the project is really trying to do	3
2.2	Why this formulation matters	4
2.3	The three recurring design motifs across all prototypes	4
3	Why the Chosen Design Makes Sense	4
3.1	Why explicit memory rather than a standard long-context transformer	4
3.2	Why cosine similarity rather than a conventional MLP classifier	4
3.3	Why two-phase training	5
3.4	Why multiple prototypes were necessary	5
4	Alternatives and When They Outperform BDH	5
4.1	Standard long-context transformer	5
4.2	Dense RAG without explicit persistent memory	5
4.3	TF-IDF or sparse lexical retrieval	5
4.4	GraphRAG	6
4.5	Hash-slot memory versus Top- k routing	6
4.6	Cross-encoder NLI-style scoring	6
5	Shared Mathematical Components	7
5.1	Hashed pseudo-tokenization	7
5.2	Encoder embedding + normalization	7
5.3	Cosine similarity score	7
5.4	Hinge-style supervision (margin separation)	7
5.5	Gated memory update primitive	8
5.6	Reconstruction regularization	8
6	V1: Dense-RAG Hybrid + Top-k Sparse Neural Memory	9
6.1	Overall Approach (Code-Faithful)	9
6.2	Representation Pipeline (Text $\rightarrow$ Vectors)	9

6.3	Top- k BDH Addressing (Sparse Neuron Activation)	9
6.4	Dense Retriever (Semantic RAG in SBERT Space)	10
6.5	Memory Module (DualStreamMemory)	10
6.5.1	Persistent State	10
6.5.2	Gated Sparse Update (Content-Driven Write)	10
6.5.3	Reconstruction Head	11
6.6	Trace Construction and Scoring	11
6.6.1	Global Trace (From Sparse Neuron Set)	11
6.6.2	Local Trace (From Retrieved Chunks)	11
6.6.3	Hybrid Score	11
6.7	Objectives (As Implemented)	11
6.7.1	Phase A: Unsupervised Ingestion Objective	11
6.7.2	Phase B: Supervised Consistency Objective	11
6.8	Handling Long Context Windows	12
6.8.1	(A) Offline long-context compression via ingestion	12
6.8.2	(B) Query-time long context via retrieval (RAG)	12
6.8.3	(C) What the code does <i>not</i> implement	12
6.9	Distinguishing Causal Signals From Noise	13
6.9.1	(1) Sparse addressing as an implicit relevance filter	13
6.9.2	(2) Gate regularization as a write-suppression prior	13
6.9.3	(3) Retrieval as an evidence-focused noise reducer	13
6.9.4	(4) Margin supervision as an implicit causal discriminator	14
6.10	Inference and Calibration	14
6.11	Limitations (Long Context + Causality Specific)	14
7	V2: Hash-Slot Memory + TF-IDF Retrieval Hybrid	15
7.1	Overall Approach (Code-Faithful)	15
7.2	Representation Pipeline (Text $\rightarrow$ Vectors)	15
7.3	Deterministic Hash-Slot Addressing (Single-Slot Access)	15
7.4	TF-IDF Retriever (Keyword RAG)	16
7.5	Memory Update (Gated Single-Slot Dynamics)	16
7.5.1	Gated update rule (as implemented)	16
7.5.2	Reconstruction head	16
7.6	Trace Construction and Dual-Stream Scoring	17
7.6.1	Global trace (entity slot)	17

7.6.2	Local trace (retrieved TF-IDF chunks)	17
7.6.3	Hybrid score	17
7.7	Objectives (As Implemented)	17
7.7.1	Phase A: Unsupervised ingestion loss	17
7.7.2	Phase B: Supervised consistency loss	17
7.8	Handling Long Context Windows	18
7.8.1	(A) Streaming compression via single-slot memory	18
7.8.2	(B) Retrieval as selective long-context expansion	18
7.8.3	(C) Claim-side long-context remains a bottleneck	18
7.9	Distinguishing Causal Signals From Noise	18
7.9.1	(1) Deterministic slotting as a hard causal bottleneck	19
7.9.2	(2) Gate regularization discourages writing noisy inputs	19
7.9.3	(3) TF-IDF retrieval acts as a lexical causal filter	19
7.10	Inference and Calibration	19
7.11	Limitations (Equation-Level, Emphasizing Long Context + Causality)	19
8	V3: BDH (No Retrieval) with Time/Location Causal Memory	21
8.1	Overall Approach (Code-Faithful)	21
8.2	Representation and Addressing Equations	21
8.2.1	Hashed pseudo-tokenization and embeddings (as in V1/V2)	21
8.2.2	Top- k BDH addressing (neural routing)	22
8.3	Time/Location Coordinate Binding (Causal Indexing)	22
8.3.1	Coordinate embeddings	22
8.3.2	Contextual fact construction (binding)	22
8.4	Sparse Update Equations (Top- k Memory Dynamics)	22
8.5	Trace Construction, Probe, and Score (No Retrieval)	23
8.5.1	Global trace from active neurons	23
8.5.2	Score	23
8.6	Objectives (As Implemented)	23
8.6.1	Phase A: ingestion objective	23
8.6.2	Phase B: supervised consistency objective (pure probe)	24
8.7	Handling Long Context Windows	24
8.7.1	(A) Sparse routing prevents quadratic growth	24
8.7.2	(B) Coordinate binding reduces drift and overwrites	24
8.7.3	(C) Memory acts as a lossy long-range compressor	24

8.8	Distinguishing Causal Signals from Noise (Non-RAG Setting)	25
8.8.1	(1) Surprise-driven gating is the implicit causal selector	25
8.8.2	(2) Forget gate implements controlled retention (noise decay)	25
8.8.3	(3) Coordinate binding discourages spurious aggregation	25
8.8.4	(4) Supervised hinge loss enforces discriminative causality	25
8.9	Inference and Calibration	26
8.10	Limitations (Equation-Level, Emphasizing Long Context + No-RAG Risks)	26
9	V4: Graph-RAG Hybrid (Knowledge Graph Retrieval) + Top-k Memory	27
9.1	Overall Approach (Code-Faithful)	27
9.2	Representation and Addressing (BDH Top- k Layer)	27
9.2.1	Hashed pseudo-tokenization and claim embedding	27
9.2.2	Top- k addressing (routing logits)	28
9.3	Graph Construction (Knowledge Graph Index)	28
9.3.1	Chunking and entity extraction	28
9.3.2	Graph definition and edge weights	28
9.3.3	Chunk incidence map	28
9.4	Graph Retrieval (1-Hop Expansion + Frequency Ranking)	29
9.4.1	Query entity extraction	29
9.4.2	1-hop neighbor expansion	29
9.4.3	Candidate chunk multiset and ranking	29
9.5	Global Memory Update (Sparse Neural Diary)	30
9.5.1	Sparse gated update	30
9.6	Trace Construction and Dual-Stream Scoring	30
9.6.1	Global trace (from neural memory)	30
9.6.2	Local trace (from retrieved graph chunks)	30
9.6.3	Mixed score (code: 0.4 / 0.6)	30
9.7	Objectives (As Implemented)	31
9.7.1	Phase A: ingestion objective	31
9.7.2	Phase B: supervised objective (margin hinge on hybrid similarity)	31
9.8	Handling Long Context Windows (Graph-RAG + Sparse Diary)	31
9.8.1	(A) Persistent global compression via Top- k memory	31
9.8.2	(B) Episodic rehydration via graph retrieval	32
9.8.3	(C) Practical truncation in code	32
9.9	Distinguishing Causal Signals from Noise (Graph-Conditioned)	32

9.9.1	(1) Graph selector: entity-conditioned retrieval suppresses narration	32
9.9.2	(2) Neural selector: surprise-driven gating writes only to active neurons . . .	32
9.9.3	(3) Supervised hinge loss determines which signals are causal for classification	33
9.10	Inference and Thresholding	33
9.11	Limitations (Equation-Level, Code-Specific)	33
10	V5: Hash-Slot CausalMemory with Time/Location Gating (Threshold-Calibrated)	35
10.1	Overall Approach (Code-Faithful)	35
10.2	Representation: From Text to Vectors	35
10.2.1	Pseudo-tokenization (hashing)	35
10.2.2	Claim embedding (bag-of-words pooling)	35
10.3	Addressing: Deterministic Hash-Slot Access	36
10.3.1	Slot index function	36
10.4	Long-Context Handling: Time/Location Conditioned Recurrent Memory	36
10.4.1	Coordinate embeddings (time and chapter)	36
10.4.2	Contextual fact construction (binding content to position)	36
10.4.3	Gated recurrent update (single-slot)	36
10.5	Trace Construction and Scoring (No Retrieval)	37
10.5.1	Entity trace via reconstruction head	37
10.5.2	Similarity score	37
10.6	Objectives (As Implemented)	37
10.6.1	Phase A: ingestion loss (reconstruction + gate regularizers)	37
10.6.2	Phase B: supervised hinge loss (consistency margin learning)	38
10.7	Threshold Calibration (Implemented as Balanced Accuracy Sweep)	38
10.7.1	Score set and decision grid	38
10.7.2	Balanced accuracy optimization	38
10.8	Handling Long Context Windows (Dedicated Discussion)	39
10.8.1	(A) Streaming update is $O(1)$ per token	39
10.8.2	(B) Time/location binding reduces repeated-token overwrites	39
10.8.3	(C) What it still cannot do	39
10.9	Distinguishing Causal Signals from Noise (Dedicated Discussion)	39
10.9.1	(1) Gating as an implicit importance filter	40
10.9.2	(2) Retention bias suppresses catastrophic drift	40
10.9.3	(3) Supervised margin defines what “causal” means for the task	40
10.10	Limitations (Equation-Level, Code-Specific)	40

11 Cross-Prototype Comparison	42
11.1 Overall Approach (Independent Hypotheses, Code-Faithful)	42
11.2 Representation (Shared Base, Explicit Differences)	42
11.3 Long-Context Handling (How Each Prototype Scales Beyond the Input Window)	43
11.4 Distinguishing Causal Signal from Noise (Mechanisms and Where Noise Leaks In)	43
11.5 Inference and Calibration (Decision Rules)	44
11.6 Key Limitations (Unified, With Prototype-Specific Emphasis)	44
12 Compact Comparison Table (Equation-Driven, Code-Faithful)	46
13 Empirical Results, Iteration Narrative, and Failure Analysis	47
13.1 Training/Validation Statistics (V1–V5)	47
13.2 Compact Summary Table (Best Checkpoints)	48
13.3 Team Iteration Narrative: Why We Started at V1 and Settled at V5	48
13.4 Why We Still Suspect Failures on This Task (Even in V5)	50
13.5 Prototype-Specific Failure Conditions (When Each Model Breaks)	50
13.5.1 V1 Failure Conditions: Dense-RAG (SBERT) + Top- k Memory	51
13.5.2 V2 Failure Conditions: Hash-Slot Memory + TF-IDF Retrieval	51
13.5.3 V3 Failure Conditions: Pure BDH + Time/Location Causal Memory (No Retrieval)	52
13.5.4 V4 Failure Conditions: Graph-RAG + Top- k Memory	52
13.5.5 V5 Failure Conditions: Hash-Slot CausalMemory + Time/Location Gating + Calibration	53
14 Primary Metrics, Calibration Logic, and Practical Target Ranges	54
14.1 Which metric matters most	54
14.2 Why balanced accuracy is the primary deployment metric	54
14.3 Why weighted F1 still matters	54
14.4 Practical target ranges	54
14.5 What a good model should look like beyond one scalar	54
15 Hyperparameter Roles and Why These Values Make Sense	55
15.1 Shared representation scale	55
15.2 Retrieval hyperparameters	55
15.3 Optimization and training schedule	56
15.4 Loss weights	56
15.5 Threshold sweep choices	56

16 Senior-Level Interview Questions and Model Answers	57
16.1 Why external memory instead of just using a larger language model?	57
16.2 Why mean pooling when you know it discards syntax?	57
16.3 Why use a reconstruction head at all?	57
16.4 Why are time and location embeddings useful?	57
16.5 Why did V5 become the preferred version?	57
16.6 Why can V2 or V5 outperform semantically richer models on some runs?	57
16.7 What is the single biggest unresolved weakness?	57
16.8 What would be the next senior-level improvement?	58
17 Conclusion	58

1 Introduction

This report documents five independent prototypes for *narrative consistency detection*. Each prototype implements a specific hypothesis about how to maintain long-term narrative state and how to query that state when classifying an entity-conditioned claim.

Task. Each sample is a natural-language claim (**content**) paired with an entity identifier (**char**). We must predict

$$y \in \{\text{consistent}, \text{contradict}\}.$$

Shared design skeleton. All prototypes share a common “similarity decision” form:

$$s(\text{char}, \text{content}) \triangleq \cos(\mathbf{r}, \mathbf{b}),$$

where:

- $\mathbf{b} \in \mathbb{R}^D$ is a claim embedding derived from **content**.
- $\mathbf{r} \in \mathbb{R}^D$ is a memory-derived trace representing entity state.

Prediction is threshold-based:

$$\hat{y} = \begin{cases} \text{consistent}, & s \geq \tau \\ \text{contradict}, & s < \tau. \end{cases}$$

What differs across prototypes. Differences are localized to:

1. **Memory addressing:** Top- k content-addressed vs deterministic hash-slot.
2. **Context acquisition:** no retrieval vs TF-IDF vs Dense (SBERT) vs Graph retrieval.
3. **Memory update dynamics:** presence/absence of time/location conditioning.
4. **Score composition:** global-only vs global+local mixture.
5. **Thresholding:** fixed threshold vs calibrated threshold sweep.

2 Project Framing and Whiteboard Walkthrough

2.1 What the project is really trying to do

At a high level, BDH On Top is not just a binary classifier. It is a structured-memory system for deciding whether a claim about an entity remains compatible with a long narrative history. The core design belief is that long-form fiction creates a *state tracking* problem, not merely a one-shot sentence classification problem. If all prior facts are compressed into a single pooled representation at decision time, early facts tend to be diluted by later narration. The project therefore externalizes persistence into an explicit memory module and asks whether the current claim geometry aligns with the remembered entity trace.

The high-level whiteboard view is:

$$\begin{aligned}
&(\text{char}, \text{content}) \rightarrow \mathbf{b} \\
&\text{novel stream} \rightarrow \mathbf{M} \\
&\mathbf{M}[\text{char}] \text{ or } \mathbf{M}[\mathcal{I}] \rightarrow \mathbf{r} \\
&s = \cos(\mathbf{r}, \mathbf{b}) \rightarrow \{\text{consistent}, \text{contradict}\}.
\end{aligned}$$

2.2 Why this formulation matters

A monolithic long-context model typically tries to solve three distinct subproblems at once: storing facts, deciding what to keep, and classifying the query. BDH separates these concerns. Sequential ingestion trains the memory mechanics; supervised margin learning trains the decision rule. That decomposition is the main technical identity of the project.

2.3 The three recurring design motifs across all prototypes

Across V1–V5, three ideas reappear:

1. **Persistent state instead of one-shot compression.** A memory matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$ stores long-range information incrementally.
2. **Similarity-space supervision instead of a deep classifier head.** The task is cast as geometric alignment between a claim embedding and a memory-derived trace.
3. **Controllability over raw expressivity.** The encoders are intentionally lightweight so that retrieval, routing, gating, and calibration effects remain interpretable.

3 Why the Chosen Design Makes Sense

3.1 Why explicit memory rather than a standard long-context transformer

The strongest conceptual decision in BDH is to externalize memory. A standard long-context encoder may outperform BDH on syntax-heavy contradiction cases, but it also hides the storage mechanism inside one opaque contextual state. BDH instead exposes the persistence mechanism directly. That makes it much easier to reason about drift, write suppression, slot collisions, retrieval mismatch, and threshold calibration.

3.2 Why cosine similarity rather than a conventional MLP classifier

Cosine similarity is intentionally simple. It forces the model to shape both memory traces and claim embeddings into a shared representation geometry. This makes the decision rule easy to inspect and threshold, although it also means lexical overlap can look falsely supportive when bag-of-words pooling collapses syntax.

3.3 Why two-phase training

The project first teaches the memory how to behave under long streams, then teaches it which distinctions matter for the downstream task. This is a more disciplined design than directly optimizing classification from the start, because direct supervision alone can push memory toward brittle shortcuts instead of stable storage.

3.4 Why multiple prototypes were necessary

The five variants are not redundant copies. Each tests a distinct hypothesis:

- V1 asks whether semantic retrieval plus sparse memory gives the best of both worlds.
- V2 asks whether deterministic entity-addressing and lexical retrieval are better aligned to the dataset.
- V3 asks whether a sufficiently stabilized memory can replace retrieval altogether.
- V4 asks whether entity-structured retrieval outperforms purely semantic retrieval.
- V5 asks whether the most stable deployment point is entity-addressed memory with time/location conditioning and calibrated thresholding.

4 Alternatives and When They Outperform BDH

4.1 Standard long-context transformer

A strong long-context transformer typically outperforms BDH when contradiction depends on syntax, negation scope, temporal qualification, or role ordering. If the context window is sufficient and compute is available, that class of model is usually better at subtle semantic reasoning than mean-pooled hashed embeddings.

BDH outperforms this alternative when the narrative is very long, explicit persistence is more important than sentence-level nuance, and the goal is to study memory mechanics rather than hide them in a larger model.

4.2 Dense RAG without explicit persistent memory

Pure dense retrieval often outperforms BDH when the relevant evidence is semantically similar but paraphrased, and when the task is mostly about locating one or two decisive chunks. It is especially strong when persistent state is not required and retrieval quality dominates performance.

BDH becomes more attractive when the system must accumulate *entity state over time* rather than repeatedly retrieve everything on demand.

4.3 TF-IDF or sparse lexical retrieval

Sparse retrieval can outperform dense retrieval when claims and evidence share named entities or discriminative lexical anchors. It is cheaper, easier to debug, and often surprisingly competitive on

structured datasets.

It underperforms when the task requires paraphrase robustness, synonym handling, or semantically similar evidence that does not share surface forms.

4.4 GraphRAG

Graph-structured retrieval can outperform both lexical and dense retrieval when the evidence is relational and entity connectivity matters more than local chunk semantics. It is useful when claims depend on social or narrative structure rather than one isolated sentence.

It underperforms when named entity recognition is noisy, aliases fragment the graph, or frequent hub nodes dominate retrieval.

4.5 Hash-slot memory versus Top- k routing

Hash-slot memory wins on simplicity, deterministic addressing, and direct entity-based probing. Top- k routing wins when one slot is too narrow to capture the relevant latent state and when distributed addressing is worth the extra training brittleness.

4.6 Cross-encoder NLI-style scoring

A cross-encoder can outperform cosine-similarity scoring when the right evidence chunk is already known, because it models pairwise interaction between claim and evidence much more directly. However, it does not natively solve the persistent memory problem and is substantially heavier at inference.

5 Shared Mathematical Components

This section defines the common primitives used throughout the prototypes.

5.1 Hashed pseudo-tokenization

For a word w , a pseudo-token id is constructed via Python hashing:

$$\text{tok}(w) = \text{hash}(w) \bmod V,$$

where V is a fixed vocabulary size (e.g., $V = 50257$).

5.2 Encoder embedding + normalization

Each token id t is embedded by a learnable table $\mathbf{E} \in \mathbb{R}^{V \times D}$:

$$\mathbf{e}(t) = \mathbf{E}[t] \in \mathbb{R}^D, \quad \mathbf{x}(t) = \text{LN}(\mathbf{e}(t)).$$

For a claim with tokens $\{t_i\}_{i=1}^L$, the claim embedding is mean-pooled:

$$\mathbf{b} = \frac{1}{L} \sum_{i=1}^L \mathbf{x}(t_i).$$

This is a bag-of-words (BoW) representation (orderless).

5.3 Cosine similarity score

For two nonzero vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^D$:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}.$$

5.4 Hinge-style supervision (margin separation)

Most prototypes implement a ReLU hinge margin in similarity space. Let $y = +1$ for **consistent**, $y = -1$ for **contradict**. For score s :

$$\mathcal{L}_{\text{sup}}(s, y) = \begin{cases} \max(0, m^+ - s), & y = +1 \\ \max(0, s - m^-), & y = -1 \end{cases}$$

In code the margins are effectively $m^+ = 0.5$ and $m^- = -0.5$, implemented as:

$$\mathcal{L}_{\text{sup}} = \begin{cases} \max(0, 0.5 - s), & y = +1 \\ \max(0, s + 0.5), & y = -1. \end{cases}$$

5.5 Gated memory update primitive

All memory modules implement a gate-based update with an input gate i and forget gate f :

$$i = \sigma(g_i([\mathbf{z}; \mathbf{m}])), \quad f = \sigma(g_f([\mathbf{z}; \mathbf{m}])),$$

$$\mathbf{m}' = f \odot \mathbf{m} + i \odot \mathbf{z}.$$

Here $\mathbf{m} \in \mathbb{R}^D$ is the current memory vector, $\mathbf{z} \in \mathbb{R}^D$ is the incoming “fact” vector, and $[\cdot; \cdot]$ denotes concatenation.

5.6 Reconstruction regularization

During ingestion, memory is trained to reconstruct the incoming fact embedding:

$$\mathbf{r} = \text{Rec}(\mathbf{m}'), \quad \mathcal{L}_{\text{recon}} = \|\mathbf{r} - \mathbf{z}\|_2^2.$$

A gate regularizer discourages indiscriminate writing:

$$\mathcal{L}_{\text{gate}} = \lambda_i \mathbb{E}(|i|) + \lambda_f \mathbb{E}(|1 - f|).$$

(Implementations differ slightly: some use only $|i|$.)

6 V1: Dense-RAG Hybrid + Top- k Sparse Neural Memory

6.1 Overall Approach (Code-Faithful)

Prototype 1 implements a **dual-stream evidence model**:

1. **Global stream (Top- k sparse neural memory)**. A persistent matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$ is updated using *content-addressed* Top- k neuron selection. This is intended to compress long narrative exposure into a stable internal state.
2. **Local stream (Dense semantic retrieval / RAG)**. A pretrained SentenceTransformer retrieves semantically relevant chunks from the novel corpus. These chunks are re-embedded in the BDH embedding space and pooled into a local trace.
3. **Hybrid decision**. Final classification is a weighted mixture of global- and local-stream cosine similarities.

Training alternates between:

- **Phase A (unlabeled ingestion)**: learn stable memory dynamics by reconstructing token embeddings after memory update.
- **Phase B (supervised consistency)**: learn a margin separator in cosine similarity space for consistent vs contradictory statements.

6.2 Representation Pipeline (Text $\rightarrow$ Vectors)

Hashed pseudo-tokenization. For each whitespace token w ,

$$t(w) = \text{hash}(w) \bmod V, \quad V = \text{CFG}[\text{'VOCAB'}].$$

Let $\mathbf{E} \in \mathbb{R}^{V \times D}$ be a trainable embedding table, and LN a LayerNorm. Then:

$$\mathbf{x}(w) = \text{LN}(\mathbf{E}[t(w)]) \in \mathbb{R}^D.$$

For a claim of T words, the claim embedding is mean pooled:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i).$$

This is the only claim representation used by the downstream decision rule.

6.3 Top- k BDH Addressing (Sparse Neuron Activation)

The encoder learns a projection tensor:

$$\mathbf{W} \in \mathbb{R}^{H \times D \times (N/H)}.$$

Given a batch tensor $\mathbf{X} \in \mathbb{R}^{B \times T \times D}$, logits are computed by:

$$\mathbf{L} = \text{einsum}(\mathbf{X}, \mathbf{W}) \in \mathbb{R}^{B \times H \times T \times (N/H)}.$$

For each (b, h, t) , Top- k indices are selected:

$$\mathcal{I}_{b,h,t} = \text{TopK}(\mathbf{L}_{b,h,t}, k), \quad k = \text{CFG}[\text{'TOP_K'}].$$

The active neuron set used for memory read/write is:

$$\mathcal{I} = \text{unique}\left(\bigcup_{h,t} \mathcal{I}_{b,h,t}\right).$$

Interpretation. $\mathcal{I}$ is a content-derived sparse address: only these neurons participate in update and probing, bounding the effective memory interaction per token.

6.4 Dense Retriever (Semantic RAG in SBERT Space)

The novels are chunked into M segments $\{c_m\}$ of fixed size (code: 120 words). A pretrained SentenceTransformer $\phi(\cdot)$ embeds each chunk:

$$\mathbf{u}_m = \phi(c_m) \in \mathbb{R}^{d_r}, \quad \mathbf{q} = \phi(q) \in \mathbb{R}^{d_r}.$$

Retrieval scores:

$$\text{score}(m) = \cos(\mathbf{q}, \mathbf{u}_m) = \frac{\mathbf{q}^\top \mathbf{u}_m}{\|\mathbf{q}\| \|\mathbf{u}_m\|}.$$

Retrieved chunk ids:

$$\mathcal{J}_K = \text{TopK}_m(\text{score}(m)).$$

Each retrieved chunk is then re-embedded into BDH space using the same hash-embedding + mean pooling pipeline:

$$\mathbf{v}_j = \frac{1}{|c_j|} \sum_{w \in c_j} \mathbf{x}(w) \in \mathbb{R}^D.$$

6.5 Memory Module (DualStreamMemory)

6.5.1 Persistent State

The model maintains a global diary:

$$\mathbf{M} \in \mathbb{R}^{N \times D}, \quad \mathbf{M} \equiv \text{global_state}.$$

6.5.2 Gated Sparse Update (Content-Driven Write)

Given an incoming fact vector $\mathbf{z} \in \mathbb{R}^D$ and active indices $\mathcal{I}$:

$$\mathbf{M}_{\mathcal{I}} \in \mathbb{R}^{|\mathcal{I}| \times D}, \quad \mathbf{Z} = \mathbf{1}_{|\mathcal{I}|} \mathbf{z}^\top.$$

Concatenate:

$$\mathbf{C} = [\mathbf{Z}; \mathbf{M}_{\mathcal{I}}] \in \mathbb{R}^{|\mathcal{I}| \times 2D}.$$

Compute gates:

$$\mathbf{i} = \sigma(g_i(\mathbf{C})) \in (0, 1)^{|\mathcal{I}| \times 1}, \quad \mathbf{f} = \sigma(g_f(\mathbf{C})) \in (0, 1)^{|\mathcal{I}| \times 1}.$$

Update each active row:

$$\mathbf{M}'_{\mathcal{I}} = \mathbf{f} \odot \mathbf{M}_{\mathcal{I}} + \mathbf{i} \odot \mathbf{Z}.$$

Only these active rows are overwritten in memory (code uses `.data` assignment).

6.5.3 Reconstruction Head

A linear reconstruction head maps memory rows back into embedding space:

$$\hat{\mathbf{Z}} = \text{Rec}(\mathbf{M}'_{\mathcal{I}}).$$

6.6 Trace Construction and Scoring

6.6.1 Global Trace (From Sparse Neuron Set)

$$\bar{\mathbf{m}} = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \mathbf{M}_j, \quad \mathbf{r}_g = \text{Rec}(\bar{\mathbf{m}}).$$

Global similarity:

$$s_g = \cos(\mathbf{r}_g, \mathbf{b}).$$

6.6.2 Local Trace (From Retrieved Chunks)

With retrieved BDH vectors $\{\mathbf{v}_j\}_{j=1}^K$:

$$\mathbf{r}_\ell = \text{WM}\left(\frac{1}{K} \sum_{j=1}^K \mathbf{v}_j\right), \quad s_\ell = \cos(\mathbf{r}_\ell, \mathbf{b}),$$

where $\text{WM}(\cdot)$ is `wm_proj` in code.

6.6.3 Hybrid Score

Validation and inference use:

$$s = 0.4 s_g + 0.6 s_\ell.$$

6.7 Objectives (As Implemented)

6.7.1 Phase A: Unsupervised Ingestion Objective

For each ingested token, after updating active rows:

$$\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{Z}} - \mathbf{Z}\|_2^2, \quad \mathcal{L}_{\text{gate}} = \mathbb{E}[\mathbf{i}],$$

$$\mathcal{L}_{\text{ingest}} = \mathcal{L}_{\text{recon}} + 0.1 \mathcal{L}_{\text{gate}}.$$

This trains: (i) stable reconstruction, (ii) conservative writing (small input gate).

6.7.2 Phase B: Supervised Consistency Objective

Let

$$s_{\text{hyb}} = \frac{s_g + s_\ell}{2}.$$

The hinge-style margin loss is:

$$\mathcal{L}_{\text{sup}}(s_{\text{hyb}}, y) = \begin{cases} \max(0, 0.5 - s_{\text{hyb}}), & y = +1 \text{ (consistent)} \\ \max(0, s_{\text{hyb}} + 0.5), & y = -1 \text{ (contradict)}. \end{cases}$$

6.8 Handling Long Context Windows

This section makes explicit how the code deals with “long context” and where it does not.

6.8.1 (A) Offline long-context compression via ingestion

The model reads a large external corpus (novels) and updates a persistent memory:

$$\mathbf{M} \leftarrow \text{Update}(\mathbf{M}, \mathbf{z}_t, \mathcal{I}_t), \quad t = 1, \dots, T_{\text{ingest}}.$$

Over many updates, the memory acts as a **streaming compressor** of narrative exposure. The capacity N and sparsity Top- k control the effective “context window”:

- N controls how many distinct memory slots can exist.
- Top- k controls how many slots are touched per token.
- Gate regularization discourages writing too aggressively, slowing drift.

Effective long-context window. Because each token only updates $|\mathcal{I}_t|$ rows, the total number of distinct rows updated after T steps is bounded by:

$$|\cup_{t=1}^T \mathcal{I}_t| \leq \min\{N, T \cdot H \cdot k\},$$

with equality reduced by deduplication. This is the core mechanism that keeps interaction sparse even when ingestion length grows.

6.8.2 (B) Query-time long context via retrieval (RAG)

At inference, the system does **not** feed long raw text into a transformer. Instead, it approximates a long context by retrieving only K chunks:

$$\{c_j\}_{j=1}^K = \text{Retrieve}(q; \{c_m\}_{m=1}^M),$$

and building a local trace $\mathbf{r}_\ell$ from them. The RAG pathway is thus a **compressed long-context window** of size $K \times 120$ words (before truncations to 50 words for BDH re-embedding).

6.8.3 (C) What the code does *not* implement

Despite the report concept of chunk pooling for **content**, this prototype does not chunk the claim and pool conditionally. The claim embedding remains:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i),$$

so any long claim is still compressed by uniform mean pooling, with no retrieval weighting inside the claim itself.

6.9 Distinguishing Causal Signals From Noise

Prototype 1 does not run explicit test-time ablations/interventions. However, it does implement two **implicit causal filters** that reduce noise influence.

6.9.1 (1) Sparse addressing as an implicit relevance filter

Only Top- k neurons are eligible to be updated:

$$\mathbf{M}'_{\mathcal{I}_t} = \mathbf{f}_t \odot \mathbf{M}_{\mathcal{I}_t} + \mathbf{i}_t \odot \mathbf{Z}_t.$$

If a token’s representation produces low logits for many neurons, it will not activate them. Thus, the Top- k operator can be interpreted as a hard attention mechanism that:

- routes “salient” content to specific memory regions,
- prevents diffuse background text from writing everywhere.

Noise failure mode. If spurious lexical patterns repeatedly trigger the same Top- k indices, noise becomes causal by repeated reinforcement (frequency bias). This is the classic failure mode of hard routing without explicit debiasing.

6.9.2 (2) Gate regularization as a write-suppression prior

The ingestion loss explicitly penalizes large input gates:

$$\mathcal{L}_{\text{gate}} = \mathbb{E}[\mathbf{i}].$$

This biases the system toward:

$$\mathbf{i} \approx 0 \quad \Rightarrow \quad \mathbf{M}' \approx \mathbf{f} \odot \mathbf{M},$$

i.e., memory retention over aggressive overwriting. In causal language, this encourages the model to treat most incoming tokens as *non-causal* unless strongly justified by the learned gating network.

6.9.3 (3) Retrieval as an evidence-focused noise reducer

The local stream selects only the top- K chunks under SBERT similarity:

$$\mathcal{J}_K = \text{TopK}_m \cos(\phi(q), \phi(c_m)).$$

This performs a strong **compression and denoising step** on the external corpus: instead of reading all chunks, it reads only those most semantically aligned with the query. This reduces irrelevant narration in the evidence set.

Noise failure mode (retrieval-induced confounding). SBERT similarity may retrieve chunks that are topically similar but label-irrelevant (high lexical/semantic overlap but wrong implication). Since the final decision weights local similarity by 0.6, retrieval noise can dominate the decision.

6.9.4 (4) Margin supervision as an implicit causal discriminator

Supervised training pushes the hybrid similarity away from ambiguous regions:

$$\mathcal{L}_{\text{sup}} = \begin{cases} \max(0, 0.5 - s_{\text{hyb}}), & y = +1 \\ \max(0, s_{\text{hyb}} + 0.5), & y = -1. \end{cases}$$

This makes the model preferentially shape memory and projections such that *features correlated with the label* become more influential in cosine space, while weakly predictive text provides inconsistent gradients and tends to be suppressed.

6.10 Inference and Calibration

Validation sweeps a threshold t to maximize weighted F1:

$$t^* = \arg \max_{t \in [-0.5, 0.5]} \text{F1}(\mathbb{I}[s \geq t], y).$$

However, inference uses a fixed threshold $\tau = 0.05$:

$$\hat{y} = \mathbb{I}[s \geq 0.05].$$

This introduces a potential calibration mismatch under distribution shift.

6.11 Limitations (Long Context + Causality Specific)

- **Claim long-context bottleneck.** Claim text is always uniformly mean pooled. There is no chunk-level retrieval pooling inside the claim, so salient subspans can be diluted:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}_i.$$

- **Retriever/classifier embedding mismatch.** Retrieval uses SBERT space, but decision uses BDH space, creating a two-space evidence alignment problem.
- **Hard Top- k routing instability.** TopK is discontinuous: small logit changes can flip indices, causing high-variance training and brittle addressing.
- **Ingestion may not encode entity-level causal facts.** Ingestion updates memory with *individual words* from novels; it primarily trains reconstruction and gating dynamics rather than explicit entity narrative state.
- **Causal confounding via lexical overlap.** Cosine similarity in a bag-of-words embedding can treat contradictory statements as similar if they share vocabulary.
- **Dominance of retrieval noise.** Since s weights local similarity more (0.6), failure in retrieval can dominate even when memory is stable.

7 V2: Hash-Slot Memory + TF-IDF Retrieval Hybrid

7.1 Overall Approach (Code-Faithful)

Prototype 2 is a **deterministic-addressing hybrid memory + keyword-RAG** system. Relative to V1:

- **Sparse addressing is replaced by deterministic hash-slot access.** Every update and every query touches *exactly one* memory row $\mathbf{M}[e]$ where $e = \text{hash}(\cdot) \bmod N$.
- **Dense semantic retrieval is replaced by TF-IDF retrieval.** Evidence is retrieved by lexical overlap (sparse dot product), not semantic similarity in an embedding space.
- **Dual-stream scoring remains.** A global trace from the hash slot and a local trace from retrieved chunks are mixed with the same weights (0.4/0.6).

Training still alternates between:

1. **Phase A: global ingestion** (unlabeled novel stream) to condition gating and reconstruction on long sequences.
2. **Phase B: supervised consistency** (train.csv) using hinge-style cosine margin separation.

7.2 Representation Pipeline (Text $\rightarrow$ Vectors)

Hashed pseudo-tokenization. For each token w :

$$t(w) = \text{hash}(w) \bmod V, \quad V = \text{CFG}[\text{'VOCAB'}].$$

Trainable embedding table $\mathbf{E} \in \mathbb{R}^{V \times D}$ with LayerNorm:

$$\mathbf{x}(w) = \text{LN}(\mathbf{E}[t(w)]) \in \mathbb{R}^D.$$

For a claim with words $\{w_i\}_{i=1}^T$, mean pooling produces:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i).$$

Note (code detail). In V2, the encoder returns only $\mathbf{X}$ (no Top- k indices). Thus, all sparsity is delegated to hash-slot addressing rather than learned routing.

7.3 Deterministic Hash-Slot Addressing (Single-Slot Access)

Slot function. For any key x (word or entity):

$$e(x) = \text{hash}(x) \bmod N, \quad N = \text{CFG}[\text{'N'}].$$

The global memory matrix is:

$$\mathbf{M} \in \mathbb{R}^{N \times D}.$$

Single-slot access is:

$$\mathbf{m}(x) = \mathbf{M}[e(x)].$$

Consequences of single-slot access. Unlike Top- k routing (V1), where $|\mathcal{I}|$ can vary and multiple rows participate in an update, V2 makes memory interaction *strictly*:

$$\text{Read/Write complexity per token} = \mathcal{O}(D),$$

but at the cost of higher collision pressure and reduced representational bandwidth per update.

7.4 TF-IDF Retriever (Keyword RAG)

The novels are chunked into M segments $\{c_m\}$ (code: 150 words per chunk). A TF-IDF vectorizer $\psi(\cdot)$ maps chunks and queries to sparse vectors in $\mathbb{R}^p$:

$$\mathbf{d}_m = \psi(c_m) \in \mathbb{R}^p, \quad \mathbf{q} = \psi(q) \in \mathbb{R}^p.$$

Scoring is sparse dot product:

$$\text{score}(m) = \mathbf{d}_m^\top \mathbf{q}.$$

Retrieved set:

$$\mathcal{J}_K = \text{TopK}_m(\text{score}(m)), \quad K = \text{top_k}.$$

Interpretation. TF-IDF retrieval is an explicit lexical overlap operator. It is robust to simple keyword matching but brittle to paraphrase and synonymy.

7.5 Memory Update (Gated Single-Slot Dynamics)

7.5.1 Gated update rule (as implemented)

For an update key x (during ingestion: $x = \text{word}$), let:

$$e = e(x), \quad \mathbf{m} = \mathbf{M}[e] \in \mathbb{R}^D, \quad \mathbf{z} = \mathbf{x}(x) \in \mathbb{R}^D.$$

Concatenate:

$$\mathbf{c} = [\mathbf{z}; \mathbf{m}] \in \mathbb{R}^{2D}.$$

Compute scalar gates (broadcast over D):

$$i = \sigma(g_i(\mathbf{c})) \in (0, 1), \quad f = \sigma(g_f(\mathbf{c})) \in (0, 1).$$

Update:

$$\mathbf{m}' = f \mathbf{m} + i \mathbf{z}.$$

Write back:

$$\mathbf{M}[e] \leftarrow \mathbf{m}'.$$

Code-level observation. Both gates are scalars per slot update (output dimension 1). This constrains the update to a global “write amount” and “forget amount” rather than feature-wise gating.

7.5.2 Reconstruction head

A linear reconstruction maps the updated slot back into embedding space:

$$\hat{\mathbf{z}} = \text{Rec}(\mathbf{m}') \in \mathbb{R}^D.$$

7.6 Trace Construction and Dual-Stream Scoring

7.6.1 Global trace (entity slot)

For an entity `char`, compute:

$$e_c = e(\text{char}) = \text{hash}(\text{char}) \bmod N,$$

$$\mathbf{r}_g = \text{Rec}(\mathbf{M}[e_c]).$$

Similarity:

$$s_g = \cos(\mathbf{r}_g, \mathbf{b}).$$

7.6.2 Local trace (retrieved TF-IDF chunks)

Retrieve chunks $\{c_j\}_{j=1}^K$ by TF-IDF. Re-embed each chunk into BDH space (hash embedding + mean pooling):

$$\mathbf{v}_j = \frac{1}{|c_j|} \sum_{w \in c_j} \mathbf{x}(w) \in \mathbb{R}^D.$$

Pool and project:

$$\mathbf{r}_\ell = \text{WM}\left(\frac{1}{K} \sum_{j=1}^K \mathbf{v}_j\right), \quad s_\ell = \cos(\mathbf{r}_\ell, \mathbf{b}).$$

7.6.3 Hybrid score

Identical mixing rule to V1:

$$s = 0.4 s_g + 0.6 s_\ell.$$

7.7 Objectives (As Implemented)

7.7.1 Phase A: Unsupervised ingestion loss

For each ingested word update, the code optimizes:

$$\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{z}} - \mathbf{z}\|_2^2, \quad \mathcal{L}_{\text{gate}} = \mathbb{E}[i],$$

$$\mathcal{L}_{\text{ingest}} = \mathcal{L}_{\text{recon}} + 0.1 \mathcal{L}_{\text{gate}}.$$

This encourages reconstruction fidelity while discouraging indiscriminate writes (small i).

7.7.2 Phase B: Supervised consistency loss

Let:

$$s_{\text{hyb}} = \frac{s_g + s_\ell}{2}.$$

With label $y \in \{+1, -1\}$ (consistent/contradict), the hinge margin is:

$$\mathcal{L}_{\text{sup}}(s_{\text{hyb}}, y) = \begin{cases} \max(0, 0.5 - s_{\text{hyb}}), & y = +1 \\ \max(0, s_{\text{hyb}} + 0.5), & y = -1. \end{cases}$$

This trains the memory and projections to separate classes in cosine space.

7.8 Handling Long Context Windows

Prototype 2 handles long context through two compression channels: (i) **persistent single-slot memory dynamics** and (ii) **keyword retrieval**.

7.8.1 (A) Streaming compression via single-slot memory

During ingestion, the model processes a long word stream $\{w_t\}_{t=1}^T$ and updates:

$$\mathbf{M}[e(w_t)] \leftarrow f_t \mathbf{M}[e(w_t)] + i_t \mathbf{x}(w_t).$$

Because each step touches one slot, the maximum number of slots influenced after T steps is:

$$\left| \{e(w_t)\}_{t=1}^T \right| \leq \min\{N, T\}.$$

Thus, V2 scales cleanly with context length (no quadratic attention). However, this comes with a **collision-limited effective context**: if many distinct words map to the same slots, the “usable” context saturates early.

Expected collision rate. If U unique keys are uniformly hashed into N slots, expected occupied slots:

$$\mathbb{E}[\#\text{occupied}] = N \left(1 - \left(1 - \frac{1}{N} \right)^U \right),$$

and expected collisions grow rapidly once U approaches N .

7.8.2 (B) Retrieval as selective long-context expansion

Instead of reading the full novel at inference, TF-IDF retrieval selects only K chunks:

$$\mathcal{J}_K = \text{TopK}_m \mathbf{d}_m^\top \mathbf{q}.$$

This produces an evidence window of roughly $150K$ words (before truncating each chunk to 50 words for BDH re-embedding). This is a strong long-context reduction, but it is **lexically gated** (depends on shared terms).

7.8.3 (C) Claim-side long-context remains a bottleneck

As in V1, the claim representation is always mean pooled:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i),$$

so long claims still suffer from dilution, and there is no internal mechanism to focus on causal spans within the claim.

7.9 Distinguishing Causal Signals From Noise

V2 lacks explicit causal attribution at inference, but it contains three implicit noise filters.

7.9.1 (1) Deterministic slotting as a hard causal bottleneck

All evidence that affects a specific slot must pass through:

$$\mathbf{M}[e] \leftarrow f \mathbf{M}[e] + i \mathbf{z}.$$

This is a strict information bottleneck: only content mapped to that slot can influence it. This can suppress diffuse noise *if* the hashing maps relevant evidence consistently to the same slot. It can also amplify noise under collisions, where irrelevant keys share the same e .

7.9.2 (2) Gate regularization discourages writing noisy inputs

Because ingestion penalizes large i :

$$\mathcal{L}_{\text{gate}} = \mathbb{E}[i],$$

the optimizer is encouraged to reduce write magnitude for common, uninformative, or inconsistent tokens. In effect, the model is biased to treat most stream tokens as non-causal unless reconstruction and training signal demand updates.

7.9.3 (3) TF-IDF retrieval acts as a lexical causal filter

TF-IDF selects evidence with maximal overlap in high-IDF terms. If the query contains discriminative entity names and rare event terms, retrieval tends to focus on causally relevant evidence. If the claim is paraphrased or uses synonyms, retrieval can miss causal evidence entirely.

Noise/confounding pattern specific to TF-IDF. High-frequency terms are downweighted by IDF, but spurious overlap on medium-frequency stylistic tokens can still dominate scores if the query lacks distinctive nouns or named entities.

7.10 Inference and Calibration

Validation tunes a threshold t by sweeping values:

$$t^* = \arg \max_{t \in [-0.5, 0.5]} \text{F1}(\mathbb{I}[s \geq t], y).$$

However, inference uses a fixed threshold:

$$\hat{y} = \mathbb{I}[s \geq 0.05],$$

which can be miscalibrated if the score distribution differs between validation and test.

7.11 Limitations (Equation-Level, Emphasizing Long Context + Causality)

- **Lexical paraphrase failure (retrieval).** TF-IDF ranks by $\mathbf{d}_m^\top \mathbf{q}$; if a paraphrase changes surface form without overlap, $\mathbf{d}_m^\top \mathbf{q} \approx 0$ and relevant chunks are not retrieved.
- **Collision-induced cross-contamination (memory).** If $\exists x \neq x'$ such that $e(x) = e(x')$, then updates intended for x overwrite x' :

$$\mathbf{M}[e] \leftarrow f \mathbf{M}[e] + i \mathbf{z}(x) \quad \Rightarrow \quad \mathbf{z}(x') \text{ is corrupted.}$$

- **Single-slot bandwidth bottleneck.** Each update can only modify one slot; multi-fact evidence that should distribute across multiple latent features is forced into a single vector:

$$\mathbf{m}' = f\mathbf{m} + i\mathbf{z}.$$

- **Scalar gating limits feature selectivity.** Gates $i, f \in \mathbb{R}$ are scalars, so the model cannot selectively update a subset of dimensions (no per-feature gating).
- **Claim BoW collapse.** Mean pooling makes contradictions with high lexical overlap appear similar:

$$\mathbf{b} = \frac{1}{T} \sum_i \mathbf{x}(w_i) \quad \text{ignores negation/order.}$$

- **Retrieval dominance risk.** With mixing $0.6 s_\ell$, failure in retrieval can dominate even if $\mathbf{r}_g$ is stable.
- **Mismatch between ingestion keys and entity keys.** Ingestion updates slots with $e(\text{word})$, while probing uses $e(\text{char})$. Unless entity identifiers appear and are ingested consistently, ingestion does not directly populate entity slots.

8 V3: BDH (No Retrieval) with Time/Location Causal Memory

8.1 Overall Approach (Code-Faithful)

Prototype 3 is the **purest** instantiation of your BDH idea in this series:

- **No retrieval channel.** There is no TF-IDF, no SBERT, no graph traversal. All evidence must be *written into* neural memory during ingestion (Phase A) and remain recoverable during probing.
- **Top- k learned addressing.** Each token dynamically activates a sparse neuron set $\mathcal{I}$ by a learned projection. This increases representational bandwidth relative to V2’s single-slot hash memory.
- **Explicit coordinate binding.** Every ingested token is bound to a time index (within-chapter word index) and a location index (chapter id). This injects a minimal notion of narrative position, crucial in non-RAG mode.
- **Single-stream scoring.** The final score is a *single* cosine similarity between a global memory trace and the claim embedding.

Training alternates:

1. **Phase A (Narrative ingestion):** sequentially read novels; update memory with time/location context; learn reconstruction + conservative write gating.
2. **Phase B (Consistency probing):** use labeled (char, content) pairs to enforce cosine-margin separation in memory space.

8.2 Representation and Addressing Equations

8.2.1 Hashed pseudo-tokenization and embeddings (as in V1/V2)

Each whitespace token w maps to a pseudo-id:

$$t(w) = \text{hash}(w) \bmod V, \quad V = \text{CFG}[\text{'VOCAB'}].$$

Embedding table $\mathbf{E} \in \mathbb{R}^{V \times D}$ and LayerNorm give:

$$\mathbf{x}(w) = \text{LN}(\mathbf{E}[t(w)]) \in \mathbb{R}^D.$$

For a claim $\{w_i\}_{i=1}^T$, mean pooled claim vector:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i) \in \mathbb{R}^D.$$

8.2.2 Top- k BDH addressing (neural routing)

Given token embedding sequence $\mathbf{X} \in \mathbb{R}^{B \times T \times D}$ and projection tensor

$$\mathbf{W} \in \mathbb{R}^{H \times D \times (N/H)},$$

the model computes routing logits:

$$\mathbf{L} = \text{einsum}(\mathbf{X}, \mathbf{W}) \in \mathbb{R}^{B \times H \times T \times (N/H)}.$$

For each (b, h, t) , choose top- k indices:

$$\mathcal{I}_{b,h,t} = \text{TopK}(\mathbf{L}_{b,h,t,:}, k).$$

The active neuron set used by memory is the flattened unique union:

$$\mathcal{I} = \text{Unique}\left(\bigcup_{h=1}^H \bigcup_{t=1}^T \mathcal{I}_{b,h,t}\right) \subseteq \{1, \dots, N\}.$$

Code-level note. In ingestion the token length is typically $T = 1$ (single word), so $\mathcal{I}$ is essentially the union across heads for that single token.

8.3 Time/Location Coordinate Binding (Causal Indexing)

8.3.1 Coordinate embeddings

For time index $t \in \mathbb{R}$ (word index within chapter) and location index $l \in \mathbb{R}$ (chapter id), the memory constructs embeddings:

$$\mathbf{t} = \tanh(\mathbf{W}_t t + \mathbf{b}_t), \quad \ell = \tanh(\mathbf{W}_\ell l + \mathbf{b}_\ell),$$

where $\mathbf{W}_t, \mathbf{W}_\ell \in \mathbb{R}^{D \times 1}$.

8.3.2 Contextual fact construction (binding)

Given a base fact vector $\mathbf{f} \in \mathbb{R}^D$ (the token embedding replicated over active indices), the bound fact is:

$$\tilde{\mathbf{f}} = \mathbf{f} \odot (\mathbf{t} + \ell).$$

Interpretation. This is multiplicative feature gating by a position-dependent signal. Two identical words at different narrative positions generate different $\tilde{\mathbf{f}}$, reducing destructive interference in memory.

8.4 Sparse Update Equations (Top- k Memory Dynamics)

Memory is a persistent matrix:

$$\mathbf{M} \in \mathbb{R}^{N \times D} \quad (\text{code: } \texttt{global_state}).$$

Let $\mathcal{I}$ be active indices for the current token/sequence. Denote current memory rows by:

$$\mathbf{M}_{\mathcal{I}} = \{\mathbf{M}_j\}_{j \in \mathcal{I}}.$$

For each active row $j \in \mathcal{I}$, compute gates (scalar per row):

$$i_j = \sigma\left(g_i\left([\tilde{\mathbf{f}}; \mathbf{M}_j]\right)\right), \quad f_j = \sigma\left(g_f\left([\tilde{\mathbf{f}}; \mathbf{M}_j]\right)\right),$$

and update:

$$\mathbf{M}'_j = f_j \mathbf{M}_j + i_j \tilde{\mathbf{f}}.$$

Writeback (as performed explicitly in code):

$$\mathbf{M}[j] \leftarrow \mathbf{M}'_j, \quad \forall j \in \mathcal{I}.$$

Why this is “state-space” in spirit. This is a discrete-time dynamical system:

$$\mathbf{M}_{t+1} = F(\mathbf{M}_t, \tilde{\mathbf{f}}_t),$$

where F is sparse and gated, and coordinate-conditioned by (t, l) .

8.5 Trace Construction, Probe, and Score (No Retrieval)

8.5.1 Global trace from active neurons

At probe time, the model obtains active indices $\mathcal{I}$ from the claim tokens (via Top- k routing), then aggregates:

$$\bar{\mathbf{m}} = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \mathbf{M}_j.$$

A reconstruction head maps to embedding space:

$$\mathbf{r} = \text{Rec}(\bar{\mathbf{m}}) \in \mathbb{R}^D.$$

8.5.2 Score

The similarity score is:

$$s = \cos(\mathbf{r}, \mathbf{b}) = \frac{\mathbf{r}^\top \mathbf{b}}{\|\mathbf{r}\| \|\mathbf{b}\|}.$$

8.6 Objectives (As Implemented)

8.6.1 Phase A: ingestion objective

For each ingested token, after update, the reconstruction is computed for each active row (broadcast target):

$$\hat{\mathbf{f}}_j = \text{Rec}(\mathbf{M}'_j), \quad j \in \mathcal{I}.$$

The code uses:

$$\mathcal{L}_{\text{recon}} = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \|\hat{\mathbf{f}}_j - \mathbf{f}\|_2^2,$$

and a gate penalty (encouraging conservative writes):

$$\mathcal{L}_{\text{gate}} = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} i_j.$$

Total ingestion loss:

$$\mathcal{L}_{\text{ingest}} = \mathcal{L}_{\text{recon}} + 0.1 \mathcal{L}_{\text{gate}}.$$

Important nuance. The reconstruction target is the *unbound* token vector $\mathbf{f}$, while memory updates use the *bound* $\tilde{\mathbf{f}} = \mathbf{f} \odot (\mathbf{t} + \ell)$. This creates a tension: memory stores contextualized facts, but reconstruction pulls it back toward context-invariant embeddings.

8.6.2 Phase B: supervised consistency objective (pure probe)

For each labeled sample, compute s via the pure probe and apply a cosine-margin hinge:

$$\mathcal{L}_{\text{sup}}(s, y) = \begin{cases} \max(0, 0.5 - s), & y = +1 \\ \max(0, s + 0.5), & y = -1. \end{cases}$$

No retrieval vectors participate; there is no auxiliary local evidence term.

8.7 Handling Long Context Windows

V3 is designed to survive long context *without* retrieval by combining three compression mechanisms.

8.7.1 (A) Sparse routing prevents quadratic growth

Instead of attending over all past tokens, each new token interacts with only $|\mathcal{I}|$ rows:

$$\text{Update cost per token} = \mathcal{O}(|\mathcal{I}|D), \quad |\mathcal{I}| \leq HkT.$$

In your ingestion loop, $T = 1$ so:

$$|\mathcal{I}| \leq Hk.$$

This is stable for arbitrarily long streams and is the primary long-context scaling argument.

8.7.2 (B) Coordinate binding reduces drift and overwrites

Two identical tokens at different narrative points yield different updates:

$$\tilde{\mathbf{f}}(t, l) = \mathbf{f} \odot (\mathbf{t}(t) + \ell(l)).$$

This reduces the “everything collapses to the same slot” phenomenon typical in plain recurrent memories. In effect, position creates additional separability in the update space.

8.7.3 (C) Memory acts as a lossy long-range compressor

After ingesting T tokens, the memory is a compressed summary:

$$\mathbf{M}_T \approx \text{Compress}(\{\tilde{\mathbf{f}}_t\}_{t=1}^T),$$

where compression is controlled by i_t and f_t . This replaces explicit long-context retrieval with persistent state.

But: coverage is still truncated in code. Because ingestion reads only `chapters[:20]` and `words[:150]`, the effective context is:

$$T_{\text{seen}} \approx 20 \times 150 \times |\text{novels}|,$$

not the full book. This is the dominant practical limitation in V3.

8.8 Distinguishing Causal Signals from Noise (Non-RAG Setting)

In V3, “causality” means: which tokens should produce lasting changes in $\mathbf{M}$ that later influence $s = \cos(\mathbf{r}, \mathbf{b})$.

8.8.1 (1) Surprise-driven gating is the implicit causal selector

The gating networks implement:

$$i_j = \sigma(g_i([\tilde{\mathbf{f}}; \mathbf{M}_j])).$$

When $\tilde{\mathbf{f}}$ is redundant given current memory, an optimal solution can drive $i_j \rightarrow 0$, suppressing writes. When it is informative, i_j increases, allowing retention. Thus i_j behaves like an implicit “causal importance” indicator.

8.8.2 (2) Forget gate implements controlled retention (noise decay)

$$\mathbf{M}'_j = f_j \mathbf{M}_j + i_j \tilde{\mathbf{f}}.$$

If f_j is systematically high for stable facts and lower for transient narration, then:

$$\mathbf{M}_j \approx \sum_{t=1}^T \left(i_{j,t} \tilde{\mathbf{f}}_t \prod_{u=t+1}^T f_{j,u} \right),$$

i.e., an exponentially-decayed accumulator. This is the formal mechanism by which non-causal noise can fade.

8.8.3 (3) Coordinate binding discourages spurious aggregation

Without (t, l) , repeated stylistic phrases push memory in the same direction. With binding, their updates differ by position:

$$\tilde{\mathbf{f}}_t = \mathbf{f}_t \odot (\mathbf{t}(t) + \ell(l)),$$

reducing the chance that purely frequent narration accumulates into a single dominant vector.

8.8.4 (4) Supervised hinge loss enforces discriminative causality

Phase B ensures that only memory content that helps separate labels is reinforced:

$$\mathcal{L}_{\text{sup}} = \max(0, 0.5 - s) \text{ or } \max(0, s + 0.5).$$

Gradients flow through $\mathbf{r} = \text{Rec}(\frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \mathbf{M}_j)$, meaning the model is pressured to store causal evidence in the rows that are *reachable* by Top- k routing from claims.

8.9 Inference and Calibration

Validation sweeps thresholds:

$$t^* = \arg \max_{t \in [-0.5, 0.5]} \text{F1}(\mathbb{I}[s \geq t], y),$$

but the prediction code uses a fixed:

$$\tau = 0.0, \quad \hat{y} = \mathbb{I}[s \geq 0].$$

Interpretation. Because cosine similarity is naturally centered near 0 for unrelated vectors, $\tau = 0$ is a reasonable default; however, any global drift in $\mathbf{r}$ or $\mathbf{b}$ shifts score distributions and can break this assumption.

8.10 Limitations (Equation-Level, Emphasizing Long Context + No-RAG Risks)

- **No retrieval fallback (hard failure mode).** If evidence is not written into rows reachable by the claim’s Top- k routing, then:

$$\mathbf{r} \text{ cannot contain the evidence} \Rightarrow s \text{ cannot recover it.}$$

- **Coverage truncation dominates quality.** Ingestion only sees a subset:

$$\{\tilde{\mathbf{f}}_t\}_{t=1}^{T_{\text{seen}}} \subset \{\tilde{\mathbf{f}}_t\}_{t=1}^{T_{\text{full}}},$$

so $\mathbf{M}$ approximates narrative statistics, not full story causality.

- **Non-differentiable Top- k routing.** The operator TopK is piecewise constant; routing changes discontinuously with logits, restricting gradient flow to selected indices.
- **Scalar gating limits feature selectivity.** $i_j, f_j \in \mathbb{R}$ scalars cannot perform dimension-wise selective retention; all D features are written/forgotten uniformly per row.
- **Contextualization–reconstruction tension.** Updates use $\tilde{\mathbf{f}} = \mathbf{f} \odot (\mathbf{t} + \ell)$ but reconstruction targets $\mathbf{f}$, encouraging memory to “undo” coordinate binding.
- **BoW claim collapse.** $\mathbf{b} = \frac{1}{T} \sum_i \mathbf{x}(w_i)$ discards order and negation, harming contradiction detection requiring compositional semantics.
- **Hash pseudo-tokenization instability.** If Python hash seeding changes, then $t(w)$ changes and the learned $\mathbf{E}[t(w)]$ no longer corresponds to the same words; checkpoints lose semantic meaning.

9 V4: Graph-RAG Hybrid (Knowledge Graph Retrieval) + Top- k Memory

9.1 Overall Approach (Code-Faithful)

Prototype 4 replaces semantic/lexical retrieval with a **knowledge-graph retrieval layer** built from **entity co-occurrence** over novel chunks, while keeping:

- **Top- k BDH addressing** for sparse neuron activation,
- **Gated global neural memory** for persistent compression,
- **Dual-stream scoring** where retrieved evidence produces a local trace mixed with the global trace.

Two memory channels in practice. The model maintains:

1. a **global diary** $\mathbf{M} \in \mathbb{R}^{N \times D}$ updated during ingestion (Phase A),
2. a **local working-memory trace** computed on-the-fly from retrieved chunks (Graph-RAG) and projected via $\text{WM}(\cdot)$.

Training phases.

1. **Phase A (Global ingestion):** sequentially read novel chapters and update $\mathbf{M}$ via Top- k addressing (no time/location conditioning in V4 code).
2. **Phase B (Supervised consistency):** use labels to push agreement between claim embedding and the *hybrid* score computed from global memory and retrieved evidence.

9.2 Representation and Addressing (BDH Top- k Layer)

9.2.1 Hashed pseudo-tokenization and claim embedding

For each word w :

$$t(w) = \text{hash}(w) \bmod V, \quad V = \text{CFG}[\text{'VOCAB'}].$$

Embedding + LayerNorm:

$$\mathbf{x}(w) = \text{LN}(\mathbf{E}[t(w)]) \in \mathbb{R}^D.$$

Claim embedding for a content sequence $\{w_i\}_{i=1}^T$:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}(w_i) \in \mathbb{R}^D.$$

9.2.2 Top- k addressing (routing logits)

Given $\mathbf{X} \in \mathbb{R}^{B \times T \times D}$ and

$$\mathbf{W} \in \mathbb{R}^{H \times D \times (N/H)},$$

routing logits:

$$\mathbf{L} = \text{einsum}(\mathbf{X}, \mathbf{W}) \in \mathbb{R}^{B \times H \times T \times (N/H)}.$$

Top- k indices:

$$\mathcal{I}_{b,h,t} = \text{TopK}(\mathbf{L}_{b,h,t,:}, k), \quad \mathcal{I} = \text{Unique}\left(\bigcup_{h,t} \mathcal{I}_{b,h,t}\right).$$

These indices drive both global memory updates and probing.

9.3 Graph Construction (Knowledge Graph Index)

9.3.1 Chunking and entity extraction

Novels are split into chunks of size S words (code uses $S = 200$). Let chunk c_m be a sequence of tokens. A NER model extracts a set of entities:

$$\mathcal{E}(c_m) = \{e_1, \dots, e_r\},$$

where in code e is included if its label is in $\{\text{PERSON}, \text{ORG}, \text{GPE}\}$.

9.3.2 Graph definition and edge weights

An undirected weighted graph $G = (V, E, w)$ is constructed:

$$V = \bigcup_m \mathcal{E}(c_m).$$

For any pair (u, v) :

$$w(u, v) = \sum_m \mathbb{I}[u \in \mathcal{E}(c_m) \wedge v \in \mathcal{E}(c_m)].$$

This is precisely what the code implements: every co-occurring entity pair increments an edge weight.

9.3.3 Chunk incidence map

Additionally, an incidence map is stored:

$$\text{Chunks}(e) = \{m : e \in \mathcal{E}(c_m)\}.$$

In code: `chunk_map[ent].append(chunk_id).`

9.4 Graph Retrieval (1-Hop Expansion + Frequency Ranking)

9.4.1 Query entity extraction

Given query text $q = (\text{char} + \text{content})$, extract:

$$\mathcal{Q} = \mathcal{E}(q).$$

If $\mathcal{Q} = \emptyset$, a fallback heuristic selects capitalized tokens:

$$\mathcal{Q} \leftarrow \{w \in q : w_1 \text{ is uppercase, } |w| > 3\}.$$

9.4.2 1-hop neighbor expansion

For each $q \in \mathcal{Q}$ present in graph, define top neighbors by edge weight:

$$\text{TopNeighbors}(q) = \arg \max_{\substack{\mathcal{S} \subseteq \mathcal{N}(q) \\ |\mathcal{S}|=J}} \sum_{v \in \mathcal{S}} w(q, v),$$

where code uses $J = 2$. Candidate entity set:

$$\mathcal{N}(\mathcal{Q}) = \mathcal{Q} \cup \bigcup_{q \in \mathcal{Q}} \text{TopNeighbors}(q).$$

9.4.3 Candidate chunk multiset and ranking

All chunks incident to any candidate entity are collected (with multiplicity):

$$\mathcal{C} = \biguplus_{v \in \mathcal{N}(\mathcal{Q})} \text{Chunks}(v).$$

The score used in code is effectively a frequency count:

$$\text{score}(m) = \sum_{v \in \mathcal{N}(\mathcal{Q})} \mathbb{I}[m \in \text{Chunks}(v)].$$

Retrieve top- K chunks:

$$\mathcal{J}_K = \text{TopK}_m(\text{score}(m)).$$

Return $\{c_m\}_{m \in \mathcal{J}_K}$.

Key property of Graph-RAG. Graph retrieval does not optimize lexical overlap or dense semantic similarity; it optimizes **structural proximity in entity co-occurrence space**. This is crucial for cross-paraphrase retrieval when the same entities are mentioned with different surface forms, but it also creates spurious proximity failure cases.

9.5 Global Memory Update (Sparse Neural Diary)

9.5.1 Sparse gated update

Global memory $\mathbf{M} \in \mathbb{R}^{N \times D}$. Let incoming token/fact vector be $\mathbf{z} \in \mathbb{R}^D$ and active indices $\mathcal{I}$ from Top- k routing.

For each $j \in \mathcal{I}$, scalar gates:

$$i_j = \sigma(g_i([\mathbf{z}; \mathbf{M}_j])), \quad f_j = \sigma(g_f([\mathbf{z}; \mathbf{M}_j])),$$

update:

$$\mathbf{M}'_j = f_j \mathbf{M}_j + i_j \mathbf{z}.$$

Writeback:

$$\mathbf{M}[j] \leftarrow \mathbf{M}'_j, \quad \forall j \in \mathcal{I}.$$

9.6 Trace Construction and Dual-Stream Scoring

9.6.1 Global trace (from neural memory)

Aggregate active rows:

$$\bar{\mathbf{m}} = \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \mathbf{M}_j, \quad \mathbf{r}_g = \text{Rec}(\bar{\mathbf{m}}).$$

Global similarity:

$$s_g = \cos(\mathbf{r}_g, \mathbf{b}).$$

9.6.2 Local trace (from retrieved graph chunks)

Each retrieved chunk c is re-embedded by BDH (not by spaCy) and pooled:

$$\mathbf{v}(c) = \frac{1}{T_c} \sum_{i=1}^{T_c} \mathbf{x}(w_i^{(c)}) \in \mathbb{R}^D.$$

Collect top- K chunk vectors $\{\mathbf{v}_1, \dots, \mathbf{v}_K\}$:

$$\mathbf{r}_\ell = \text{WM}\left(\frac{1}{K} \sum_{k=1}^K \mathbf{v}_k\right), \quad s_\ell = \cos(\mathbf{r}_\ell, \mathbf{b}).$$

9.6.3 Mixed score (code: 0.4 / 0.6)

Final score:

$$s = \alpha s_g + (1 - \alpha) s_\ell, \quad \alpha = 0.4.$$

9.7 Objectives (As Implemented)

9.7.1 Phase A: ingestion objective

Given reconstruction head output on updated active rows:

$$\hat{\mathbf{z}}_j = \text{Rec}(\mathbf{M}'_j),$$

the code uses:

$$\begin{aligned}\mathcal{L}_{\text{recon}} &= \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \|\hat{\mathbf{z}}_j - \mathbf{z}\|_2^2, & \mathcal{L}_{\text{gate}} &= \frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} i_j, \\ \mathcal{L}_{\text{ingest}} &= \mathcal{L}_{\text{recon}} + 0.1 \mathcal{L}_{\text{gate}}.\end{aligned}$$

9.7.2 Phase B: supervised objective (margin hinge on hybrid similarity)

In training loop, the code uses:

$$s_{\text{hyb}} = \frac{s_g + s_\ell}{2}.$$

Then hinge-margin with labels $y \in \{+1, -1\}$:

$$\mathcal{L}_{\text{sup}}(s_{\text{hyb}}, y) = \begin{cases} \max(0, 0.5 - s_{\text{hyb}}), & y = +1 \\ \max(0, s_{\text{hyb}} + 0.5), & y = -1. \end{cases}$$

Note the mismatch between training and validation/inference. Validation/inference uses

$$s = 0.4s_g + 0.6s_\ell,$$

while training uses $(s_g + s_\ell)/2$. This can create calibration drift between what is optimized and what is scored.

9.8 Handling Long Context Windows (Graph-RAG + Sparse Diary)

Prototype 4 handles long context through **two complementary compression paths**.

9.8.1 (A) Persistent global compression via Top- k memory

The global diary is a streaming compressor:

$$\mathbf{M}_{t+1} = F(\mathbf{M}_t, \mathbf{z}_t), \quad \text{with sparse updates on } \mathcal{I}_t.$$

The complexity per token is:

$$\mathcal{O}(|\mathcal{I}_t|D), \quad |\mathcal{I}_t| \leq HkT.$$

Thus, global memory supports arbitrarily long input streams without quadratic attention growth.

9.8.2 (B) Episodic rehydration via graph retrieval

Graph-RAG acts as an external episodic store. Instead of needing the global diary to preserve every detail, the model can “rehydrate” relevant context by retrieving chunks:

$$q \rightarrow \mathcal{Q} \rightarrow \mathcal{N}(\mathcal{Q}) \rightarrow \mathcal{J}_K \rightarrow \{c_m\}.$$

This is particularly valuable when:

- the global diary has drifted or overwritten facts,
- the evidence is rare or appears sparsely,
- the claim requires relational context (multiple entities) not preserved in a single memory trace.

9.8.3 (C) Practical truncation in code

As with other prototypes, ingestion reads only:

$$15 \text{ chapters} \times 100 \text{ words per chapter},$$

so the diary is not a complete narrative memory. Graph-RAG partially compensates by indexing the *entire* novels into graph chunks.

9.9 Distinguishing Causal Signals from Noise (Graph-Conditioned)

In V4, the system distinguishes causal signals from noise via **two selectors**:

9.9.1 (1) Graph selector: entity-conditioned retrieval suppresses narration

Retrieval is driven by entity sets $\mathcal{Q}$ and their 1-hop neighborhoods. Chunks not incident to these nodes are never considered. Formally, the retrieval operator restricts evidence to:

$$\{c_m : \exists v \in \mathcal{N}(\mathcal{Q}) \text{ s.t. } m \in \text{Chunks}(v)\}.$$

This filters large volumes of stylistic narration (noise) that lacks salient entities.

9.9.2 (2) Neural selector: surprise-driven gating writes only to active neurons

Even within ingested or retrieved text, memory updates are gated:

$$\mathbf{M}'_j = f_j \mathbf{M}_j + i_j \mathbf{z}.$$

An optimal solution can push $i_j \rightarrow 0$ for redundant/high-frequency narration and increase i_j for informative tokens, making gating an implicit causal importance indicator.

9.9.3 (3) Supervised hinge loss determines which signals are causal for classification

The supervised loss couples the hybrid similarity to labels:

$$\partial \mathcal{L}_{\text{sup}} / \partial s_{\text{hyb}}$$

drives both:

- what the global diary must retain (to raise s_g for consistent claims),
- what the working-memory projection should emphasize (to raise s_ℓ when retrieval is correct).

This is an implicit causality filter: only evidence that improves discrimination receives consistent gradients.

9.10 Inference and Thresholding

Validation sweeps thresholds t and picks the best F1, but prediction uses a fixed:

$$\tau = 0.05, \quad \hat{y} = \mathbb{I}[s \geq \tau].$$

Given the mixture weighting, the score distribution depends heavily on retrieval quality; τ is therefore less stable than in pure cosine settings.

9.11 Limitations (Equation-Level, Code-Specific)

- **NER noise corrupts G and $\mathcal{Q}$.** Let the NER extractor be $\hat{\mathcal{E}}(\cdot)$. Errors imply:

$$\hat{\mathcal{E}}(c_m) \neq \mathcal{E}(c_m) \Rightarrow \hat{V} \neq V, \hat{w} \neq w, \hat{\mathcal{Q}} \neq \mathcal{Q}.$$

This perturbs neighbor expansion and can delete the true evidence path.

- **Spurious co-occurrence does not equal factual relevance.** Edge weights capture proximity, not entailment:

$$w(u, v) \text{ large} \not\Rightarrow u \text{ and } v \text{ are causally related facts.}$$

Thus retrieval can return “socially close” but irrelevant chunks.

- **Alias and surface-form fragmentation.** Entities are stored as raw strings; two mentions of the same entity (e.g., “Edmond Dantès” vs “Dantès”) create distinct nodes:

$$e \neq e' \Rightarrow \text{Chunks}(e) \cap \text{Chunks}(e') \text{ may be small,}$$

hurting recall.

- **Degree bias and hub domination.** High-degree nodes (frequent places/titles) can dominate neighbor expansion. If q is a hub:

$$|\mathcal{N}(q)| \text{ large and semantically broad} \Rightarrow \text{retrieval becomes noisy.}$$

- **Training–inference mismatch in mixture form.** Training uses:

$$s_{\text{hyb}} = \frac{s_g + s_\ell}{2},$$

while validation/inference use:

$$s = 0.4s_g + 0.6s_\ell.$$

This mismatch can lead to suboptimal calibration and unstable thresholds.

- **Top- k non-differentiability.** The routing operator $\text{TopK}(\cdot)$ is discontinuous, limiting gradients through addressing.
- **BoW pooling collapses syntax and negation.** Both claim and retrieved chunks are pooled:

$$\mathbf{b} = \frac{1}{T} \sum_i \mathbf{x}(w_i), \quad \mathbf{v}(c) = \frac{1}{T_c} \sum_i \mathbf{x}(w_i^{(c)}),$$

so contradictions depending on compositional structure can be missed even if retrieval is correct.

10 V5: Hash-Slot CausalMemory with Time/Location Gating (Threshold-Calibrated)

10.1 Overall Approach (Code-Faithful)

Prototype 5 is the most “structured-memory-first” variant: it removes retrieval entirely and replaces Top- k addressing with **deterministic hash-slot addressing**, while adding **explicit time/location conditioning** inside the memory update. The decision rule is **not fixed**; it is **calibrated** by sweeping a threshold on training scores to maximize balanced accuracy.

Key design choices.

- **Encoder:** word hashing $\rightarrow$ embedding table $\rightarrow$ LayerNorm (a lightweight BDH-style encoder; the decoder tensor exists but is unused for routing here).
- **Memory:** a persistent state matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$ updated via **scalar input/forget gates** and a **reconstruction head**.
- **Context binding:** every ingested token embedding is multiplicatively modulated by **time index** and **chapter index** embeddings.
- **Supervision:** hinge-margin alignment between the **entity slot** and the **claim embedding**.
- **Threshold selection:** choose τ^* on training data by maximizing balanced accuracy and then reuse it for test predictions.

10.2 Representation: From Text to Vectors

10.2.1 Pseudo-tokenization (hashing)

Each word w is mapped to a pseudo-token id:

$$t(w) = \text{hash}(w) \bmod V, \quad V = \text{VOCAB_SIZE}.$$

The encoder produces a normalized embedding:

$$\mathbf{f}(w) = \text{LN}(\mathbf{E}[t(w)]) \in \mathbb{R}^D.$$

In code, `BDH_GPU.forward` returns $\mathbf{f}(w)$ (the computed tensor k is unused downstream).

10.2.2 Claim embedding (bag-of-words pooling)

For a claim (content) tokenized into words $\{w_i\}_{i=1}^T$, the code embeds each word independently and mean-pools:

$$\mathbf{b} = \frac{1}{T} \sum_{i=1}^T \mathbf{f}(w_i) \in \mathbb{R}^D.$$

10.3 Addressing: Deterministic Hash-Slot Access

10.3.1 Slot index function

Prototype 5 addresses memory slots deterministically via:

$$e(x) = \text{hash}(x) \bmod N.$$

Two different keys are used:

- **Ingestion (novels):** $e_t = e(w_t)$ (word-indexed slots),
- **Supervised probing:** $e_c = e(\text{char})$ (entity-indexed slots).

Collision condition. For $x \neq x'$,

$$e(x) = e(x') \Rightarrow \text{interference in the same slot } \mathbf{M}_{e(x)}.$$

10.4 Long-Context Handling: Time/Location Conditioned Recurrent Memory

10.4.1 Coordinate embeddings (time and chapter)

During ingestion, each token has:

- time index t = word position inside the sampled chapter segment,
- location index l = chapter index inside the sampled chapter subset.

The memory module computes coordinate embeddings:

$$\mathbf{t} = \tanh(\mathbf{W}_t t + \mathbf{b}_t), \quad \mathbf{\ell} = \tanh(\mathbf{W}_\ell l + \mathbf{b}_\ell),$$

where $\mathbf{W}_t, \mathbf{W}_\ell \in \mathbb{R}^{D \times 1}$.

10.4.2 Contextual fact construction (binding content to position)

Let the raw token embedding be $\mathbf{f}_t = \mathbf{f}(w_t)$. The contextualized fact is:

$$\tilde{\mathbf{f}}_t = \mathbf{f}_t \odot (\mathbf{t} + \mathbf{\ell}).$$

This makes identical words behave differently at different positions/chapters, which is the primary long-context stabilization mechanism in this prototype.

10.4.3 Gated recurrent update (single-slot)

Let the current slot vector be $\mathbf{m} = \mathbf{M}_{e_t} \in \mathbb{R}^D$. Gates are computed from the concatenation of contextual fact and current memory:

$$i = \sigma(g_i([\tilde{\mathbf{f}}_t; \mathbf{m}])), \quad f = \sigma(g_f([\tilde{\mathbf{f}}_t; \mathbf{m}])),$$

where $i, f \in (0, 1)$ are *scalars* (code outputs shape $[1, 1]$). Update:

$$\mathbf{m}' = f \mathbf{m} + i \tilde{\mathbf{f}}_t.$$

Writeback:

$$\mathbf{M}_{e_t} \leftarrow \mathbf{m}'.$$

Why this supports long context (mechanistically). Over long streams, position binding reduces destructive overwrites by ensuring repeated high-frequency tokens map to different effective write vectors:

$$\tilde{\mathbf{f}}_t \neq \tilde{\mathbf{f}}_{t'} \text{ even if } \mathbf{f}_t = \mathbf{f}_{t'}.$$

This is not a full sequence model, but it is a stabilizer for recurrent writes.

10.5 Trace Construction and Scoring (No Retrieval)

10.5.1 Entity trace via reconstruction head

For a supervised query with entity key $e_c = e(\text{char})$, the trace is:

$$\mathbf{r} = \text{Rec}(\mathbf{M}_{e_c}), \quad \text{Rec}(\cdot) = \text{reconstruction_head}.$$

10.5.2 Similarity score

Given claim embedding $\mathbf{b}$:

$$s = \cos(\mathbf{r}, \mathbf{b}) = \frac{\mathbf{r}^\top \mathbf{b}}{\|\mathbf{r}\|_2 \|\mathbf{b}\|_2}.$$

Prediction uses a calibrated threshold τ^* :

$$\hat{y} = \mathbb{I}[s \geq \tau^*], \quad \hat{y} = 1 \Rightarrow \text{consistent}, \quad \hat{y} = 0 \Rightarrow \text{contradict}.$$

10.6 Objectives (As Implemented)

10.6.1 Phase A: ingestion loss (reconstruction + gate regularizers)

The memory returns a reconstructed vector:

$$\hat{\mathbf{f}}_t = \text{Rec}(\mathbf{m}').$$

The code defines:

$$\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{f}}_t - \mathbf{f}_t\|_2^2.$$

Two regularizers are applied:

$$\mathcal{L}_{\text{gate}} = \mathbb{E}[|i|], \quad \mathcal{L}_{\text{decay}} = \mathbb{E}[|1 - f|].$$

So the ingestion objective is:

$$\mathcal{L}_{\text{ingest}} = W_{\text{recon}} \mathcal{L}_{\text{recon}} + W_{\text{gating}} \mathcal{L}_{\text{gate}} + W_{\text{decay}} \mathcal{L}_{\text{decay}},$$

with code weights:

$$W_{\text{recon}} = 1.0, \quad W_{\text{gating}} = 0.1, \quad W_{\text{decay}} = 0.01.$$

Interpretation.

- Minimizing $\mathcal{L}_{\text{recon}}$ forces memory to preserve information sufficient to reconstruct embeddings.
- Minimizing $\mathcal{L}_{\text{gate}}$ encourages conservative writes ($i \rightarrow 0$ on average), mitigating drift.
- Minimizing $\mathcal{L}_{\text{decay}}$ encourages retention ($f \rightarrow 1$), again mitigating overwrites.

10.6.2 Phase B: supervised hinge loss (consistency margin learning)

For each labeled pair $(\text{char}, \text{content}, y)$, score $s = \cos(\mathbf{r}, \mathbf{b})$ and apply:

$$\mathcal{L}_{\text{sup}}(s, y) = \begin{cases} \max(0, 0.5 - s), & y = +1 \text{ (consistent)} \\ \max(0, s + 0.5), & y = -1 \text{ (contradict)}. \end{cases}$$

Batch sum is scaled:

$$\mathcal{L}_{\text{cons}} = W_{\text{cons}} \sum \mathcal{L}_{\text{sup}}, \quad W_{\text{cons}} = 2.0.$$

What supervision is actually shaping. Supervision does *not* directly supervise ingestion slots $e(w)$. It supervises only entity slots $e(\text{char})$ through $\mathbf{M}_{e_c}$ and $\text{Rec}(\cdot)$, which is why entity/ingestion mismatch remains a central limitation.

10.7 Threshold Calibration (Implemented as Balanced Accuracy Sweep)

10.7.1 Score set and decision grid

Compute training scores:

$$s_n = \cos(\text{Rec}(\mathbf{M}_{e_c}(\text{char}_n)), \mathbf{b}_n), \quad y_n \in \{0, 1\}.$$

Sweep thresholds over a grid $\mathcal{T}$ (code: $\mathcal{T} = \{-1, -1 + \Delta, \dots, 1\}$ with 100 points):

$$\hat{y}_n(\tau) = \mathbb{I}[s_n \geq \tau].$$

10.7.2 Balanced accuracy optimization

Balanced accuracy:

$$\text{BalAcc}(\tau) = \frac{1}{2}(\text{TPR}(\tau) + \text{TNR}(\tau)).$$

Choose:

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \text{BalAcc}(\tau).$$

This yields a **data-driven** threshold for the particular score distribution induced by the current encoder+memory state.

Why this matters relative to fixed-threshold prototypes. If score distributions drift due to:

- different training duration,
- different gate regularization,
- different collision patterns,
- different content length distribution,

then a fixed τ becomes brittle. Calibrating τ^* partially absorbs such shifts.

10.8 Handling Long Context Windows (Dedicated Discussion)

Prototype 5 supports long context primarily through **online compression** rather than retrieval.

10.8.1 (A) Streaming update is $O(1)$ per token

Each token updates exactly one slot:

$$\mathbf{M}_{e_t} \leftarrow f\mathbf{M}_{e_t} + i\tilde{\mathbf{f}}_t,$$

with cost $\mathcal{O}(D)$. This avoids quadratic attention and scales to long sequences.

10.8.2 (B) Time/location binding reduces repeated-token overwrites

Repeated words across chapters do not create identical write vectors; instead:

$$\tilde{\mathbf{f}}_t = \mathbf{f}_t \odot (\mathbf{t} + \ell)$$

forces the memory to represent “*the same word at different narrative coordinates*” as different signals. This is a practical heuristic for long-form text where the same surface token appears in many contexts.

10.8.3 (C) What it still cannot do

This is not a relational store. If the narrative fact is multi-entity (“X betrayed Y”), the memory slot is keyed by a single hash (a word slot during ingestion, or an entity slot during probing). Without explicit relation keys, long-context can be retained but not reliably *addressed*.

10.9 Distinguishing Causal Signals from Noise (Dedicated Discussion)

Prototype 5 distinguishes signal from noise via **gates + supervision** (no retrieval filter exists).

10.9.1 (1) Gating as an implicit importance filter

The input gate i controls write magnitude. Under the joint objective:

$$\min \mathcal{L}_{\text{ingest}} + \mathcal{L}_{\text{cons}},$$

the optimizer can reduce writes for frequent but non-discriminative tokens by driving $i \rightarrow 0$, because:

$$\text{unhelpful writes} \Rightarrow \text{increase drift} \Rightarrow \text{worse supervised margin}.$$

10.9.2 (2) Retention bias suppresses catastrophic drift

The decay regularizer pushes $f \rightarrow 1$:

$$\mathcal{L}_{\text{decay}} = \mathbb{E}[|1 - f|],$$

which biases the model to preserve existing slot content unless the token strongly improves reconstruction or consistency performance.

10.9.3 (3) Supervised margin defines what “causal” means for the task

For consistent samples ($y = +1$), gradients increase similarity:

$$\frac{\partial \mathcal{L}_{\text{sup}}}{\partial s} < 0 \quad \Rightarrow \quad \text{pull } \mathbf{r} \text{ toward } \mathbf{b}.$$

For contradictory samples ($y = -1$), gradients decrease similarity:

$$\frac{\partial \mathcal{L}_{\text{sup}}}{\partial s} > 0 \quad \Rightarrow \quad \text{push } \mathbf{r} \text{ away from } \mathbf{b}.$$

Thus, “causal signal” is defined operationally as whatever improves the margin separation in cosine space.

10.10 Limitations (Equation-Level, Code-Specific)

- **Entity/ingestion mismatch (core structural issue).** Ingestion writes to $e(w)$ while supervision probes $e(\text{char})$:

$$e(w) \neq e(\text{char}) \Rightarrow \mathbf{M}_{e(\text{char})} \text{ may remain uninitialized by narrative facts.}$$

This means ingestion mainly trains gating mechanics and reconstruction stability, not entity knowledge population.

- **BoW collapse (syntax/negation failure).** Claim embedding:

$$\mathbf{b} = \frac{1}{T} \sum_i \mathbf{f}(w_i)$$

discards order and scope, so:

$$\text{“X is alive”} \approx \text{“X is not alive”}$$

whenever overlap dominates.

- **Hash collisions cause persistent interference.** If $e(x) = e(x')$ then updates entangle:

$$\mathbf{M}_{e(x)} \leftarrow f\mathbf{M}_{e(x)} + i\tilde{\mathbf{f}}(x) \quad \text{contaminates} \quad \mathbf{M}_{e(x')}.$$

- **Time/location embeddings are not grounded to entities.** Coordinates modulate token writes but do not enforce entity binding:

$$\tilde{\mathbf{f}}_t = \mathbf{f}_t \odot (\mathbf{t} + \boldsymbol{\ell})$$

still writes to $e(w_t)$, not to an entity key.

- **Threshold calibration can overfit training distribution.** Selecting τ^* via:

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \text{BalAcc}(\tau)$$

may not transfer if test score distributions shift (different content lengths, different entity frequencies). This is especially relevant because the score is a cosine between a reconstructed trace and a pooled embedding, both sensitive to distributional changes.

- **Compute inefficiency and optimizer noise.** Per-word optimizer steps yield high-variance gradients and underutilize GPU parallelism; this encourages truncation (10 chapters $\times$ 150 words) and biases the learned memory dynamics to early narrative segments.

11 Cross-Prototype Comparison

11.1 Overall Approach (Independent Hypotheses, Code-Faithful)

- **P1 (Dense-RAG + Top- k memory)** tests: *evidence grounding improves when a semantic retriever supplies relevant narrative chunks, while Top- k memory provides a persistent global trace.* In code, memory is *content-addressed by the claim tokens* (no explicit entity slot).
- **P2 (TF-IDF + Hash-slot memory)** tests: *lexical retrieval is sufficient when key terms overlap; deterministic addressing improves stability.* In code, memory is *entity-addressed* for probing via $e(\text{char})$.
- **P3 (Pure BDH + time/location causal memory)** tests: *retrieval can be removed if the memory update is stabilized by explicit narrative coordinates.* In code, memory is *content-addressed by the claim tokens* (no entity slot), but ingestion is enriched by time/location.
- **P4 (Graph-RAG + Top- k memory)** tests: *entity co-occurrence structure is a useful retrieval prior when semantic retrieval is unavailable/unstable.* In code, graph retrieval is entity-centric, but the memory probe remains *content-addressed*.
- **P5 (Hash-slot CausalMemory + time/location gating + calibration)** tests: *structured recurrent memory dynamics plus threshold calibration can compete without retrieval.* In code, probing is *entity-addressed* and the decision threshold is *learned by sweep*.

11.2 Representation (Shared Base, Explicit Differences)

All prototypes construct token embeddings from a hashed pseudo-tokenization:

$$t(w) = \text{hash}(w) \bmod V, \quad \mathbf{x}_i = \text{LN}(\mathbf{E}[t(w_i)]) \in \mathbb{R}^D, \quad \mathbf{b} = \frac{1}{L} \sum_{i=1}^L \mathbf{x}_i.$$

The principal differences are in (i) **memory addressing**, (ii) **retrieval operator**, and (iii) **trace construction**.

Memory addressing.

- **Top- k content addressing (P1, P3, P4):**

$$\mathbf{L} = \text{einsum}(\mathbf{X}, \mathbf{W}) \in \mathbb{R}^{B \times H \times T \times (N/H)}, \quad \mathcal{I} = \text{Unique}(\text{TopK}(\mathbf{L}, k)).$$

- **Hash-slot addressing (P2, P5):**

$$e(x) = \text{hash}(x) \bmod N, \quad \mathbf{m} = \mathbf{M}_{e(x)}.$$

Retrieval operator (when present).

- **Dense semantic retrieval (P1):**

$$\text{score}(c) = \cos(\phi(q), \phi(c)).$$

- **TF-IDF lexical retrieval (P2):**

$$\text{score}(c) = \psi(q)^\top \psi(c).$$

- **Graph-RAG retrieval (P4):**

$$\text{score}(c_m) = \sum_{v \in \mathcal{N}(\mathcal{Q})} \mathbb{I}[m \in \text{Chunks}(v)], \quad \mathcal{N}(\mathcal{Q}) = \mathcal{Q} \cup \bigcup_{q \in \mathcal{Q}} \text{TopNeighbors}(q).$$

11.3 Long-Context Handling (How Each Prototype Scales Beyond the Input Window)

Streaming compression into persistent memory (all prototypes). Each prototype implements a recurrent write into a persistent memory matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$ using gates:

$$i = \sigma(g_i([\mathbf{z}; \mathbf{m}])), \quad f = \sigma(g_f([\mathbf{z}; \mathbf{m}])), \quad \mathbf{m}' = f \odot \mathbf{m} + i \odot \mathbf{z}.$$

The key difference is *which rows are updated* and whether $\mathbf{z}$ is coordinate-conditioned.

Selective write bandwidth.

- **P1/P3/P4 (Top- k)** update only $\mathbf{M}_{\mathcal{I}}$, providing a controllable write bandwidth (content-driven sparsity).
- **P2/P5 (Hash-slot)** update exactly one row per step, yielding $\mathcal{O}(D)$ write cost and deterministic access.

Coordinate conditioning for long narratives (P3, P5). These prototypes explicitly bind token embeddings to narrative coordinates (time t , chapter/location l):

$$\mathbf{t} = \tanh(\mathbf{W}_t t + \mathbf{b}_t), \quad \boldsymbol{\ell} = \tanh(\mathbf{W}_\ell l + \mathbf{b}_\ell), \quad \tilde{\mathbf{z}} = \mathbf{z} \odot (\mathbf{t} + \boldsymbol{\ell}),$$

then write with $\mathbf{z} \leftarrow \tilde{\mathbf{z}}$. This reduces repeated-token overwrite drift across long contexts.

Expansion via retrieval (P1, P2, P4). Retrieval functions as an external long-context expander: it selects a small set of chunks $\{c_j\}_{j=1}^K$ and re-embeds them to form a local trace. This compensates for ingestion truncation and imperfect memory writes, but introduces retrieval-specific failure modes.

11.4 Distinguishing Causal Signal from Noise (Mechanisms and Where Noise Leaks In)

Across prototypes, causal filtering is largely *implicit* via the same composite objective family:

$$\mathcal{L} = \underbrace{\|\text{Rec}(\mathbf{m}') - \mathbf{z}\|_2^2}_{\mathcal{L}_{\text{recon}}} + \lambda_i \underbrace{\mathbb{E}(|i|)}_{\text{conservative write}} + \lambda_f \underbrace{\mathbb{E}(|1 - f|)}_{\text{retention bias}} + \beta \underbrace{\mathcal{L}_{\text{sup}}}_{\text{margin alignment}}.$$

Additionally, retrieval can act as a *noise filter* (when it succeeds) by reducing exposure to irrelevant narration:

- **P1:** semantic retrieval reduces background noise when $\cos(\phi(q), \phi(c))$ correlates with factual relevance.
- **P2:** TF-IDF reduces noise only under lexical overlap; it can amplify spurious cues (name overlap, repeated phrases).
- **P4:** graph retrieval biases evidence toward entity neighborhoods; it can also import “socially adjacent” but causally irrelevant chunks via co-occurrence.
- **P3/P5:** no retrieval; noise control relies entirely on gates (i, f) and the supervised margin shaping the memory manifold.

11.5 Inference and Calibration (Decision Rules)

Scoring forms.

- **Dual-stream (P1, P2, P4):**

$$s = \alpha \cos(\mathbf{r}_g, \mathbf{b}) + (1 - \alpha) \cos(\mathbf{r}_\ell, \mathbf{b}), \quad \alpha = 0.4 \text{ (code)}.$$

- **Pure global (P3):**

$$s = \cos(\mathbf{r}, \mathbf{b}).$$

- **Entity-slot cosine (P5):**

$$s = \cos(\text{Rec}(\mathbf{M}_{e(\text{char})}), \mathbf{b}).$$

Calibration.

- **Fixed thresholds (P1/P2/P4):** $\hat{y} = \mathbb{I}[s \geq \tau]$ with $\tau = 0.05$ (code).
- **Fixed threshold (P3):** $\tau = 0.0$ (code; cosine-centered assumption).
- **Calibrated threshold (P5):**

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \text{BalAcc}(\{\mathbb{I}[s_n \geq \tau]\}, \{y_n\}).$$

11.6 Key Limitations (Unified, With Prototype-Specific Emphasis)

- **BoW embedding collapse (all):**

$$\mathbf{b} = \frac{1}{L} \sum_i \mathbf{x}_i$$

drops order, negation scope, and role structure, limiting contradiction sensitivity.

- **Hash collisions (P2, P5; also present indirectly via hashed token ids in others):**

$$e(x) = e(x') \Rightarrow \mathbf{M}_{e(x)} \text{ is a shared interference channel.}$$

- **Addressing semantics mismatch (code-critical):**

- **P1/P3/P4:** probing is *claim-content-addressed* (via $\mathcal{I}$), not entity-addressed; **char** influences retrieval only (P1/P4) or not at all (P3).
- **P2/P5:** ingestion updates $e(w)$ while supervision probes $e(\mathbf{char})$, so entity slots are not guaranteed to be populated by narrative facts.
- **Retriever-specific failures (P1/P2/P4):**
 - Dense (P1): embedding-space mismatch between $\phi(\cdot)$ and BDH space; compute overhead.
 - TF-IDF (P2): paraphrase blindness; lexical confounds dominate $\psi(q)^\top \psi(c)$.
 - Graph (P4): NER aliasing/omissions corrupt $\mathcal{Q}$ and $\mathcal{E}(c)$; co-occurrence confounds do not imply factual relevance.
- **Ingestion truncation (all):** per-token loops restrict coverage, biasing what the memory actually sees and retains.

12 Compact Comparison Table (Equation-Driven, Code-Faithful)

Proto	Memory Keying / Addressing	Retrieval Score	Coord. Condi- tioning	Final Score + Calibra- tion
P1	Top- k content addressing: $\mathcal{I} = \text{TopK}(\text{einsum}(\mathbf{X}, \mathbf{W}))$ (Probe uses claim $\mathcal{I}$)	Dense: $\cos(\phi(q), \phi(c))$	None	$s = 0.4 \cos(\mathbf{r}_g, \mathbf{b}) + 0.6 \cos(\mathbf{r}_\ell, \mathbf{b}); \tau = 0.05$
P2	Hash-slot: $e(x) = \text{hash}(x) \bmod N$ (Probe uses $e(\text{char})$)	TF-IDF: $\psi(q)^\top \psi(c)$	None	$s = 0.4 \cos(\mathbf{r}_g, \mathbf{b}) + 0.6 \cos(\mathbf{r}_\ell, \mathbf{b}); \tau = 0.05$
P3	Top- k content addressing: $\mathcal{I} = \text{TopK}(\text{einsum}(\mathbf{X}, \mathbf{W}))$ (Probe uses claim $\mathcal{I}$)	None	Yes: $\tilde{\mathbf{z}} = \mathbf{z} \odot (\mathbf{t} + \ell)$	$s = \cos(\mathbf{r}, \mathbf{b}); \tau = 0.0$
P4	Top- k content addressing: $\mathcal{I} = \text{TopK}(\text{einsum}(\mathbf{X}, \mathbf{W}))$ (Probe uses claim $\mathcal{I}$)	Graph: $\sum_{v \in \mathcal{N}(\mathcal{Q})} \mathbb{I}[c \in \text{Chunks}(v)]$	None	$s = 0.4 \cos(\mathbf{r}_g, \mathbf{b}) + 0.6 \cos(\mathbf{r}_\ell, \mathbf{b}); \tau = 0.05$
P5	Hash-slot: $e(x) = \text{hash}(x) \bmod N$ (Probe uses $e(\text{char})$)	None	Yes: $\tilde{\mathbf{f}} = \mathbf{f} \odot (\mathbf{t} + \ell)$	$s = \cos(\text{Rec}(\mathbf{M}_{e(\text{char})}), \mathbf{b}); \tau^* = \arg \max_\tau \text{BalAcc}$

13 Empirical Results, Iteration Narrative, and Failure Analysis

13.1 Training/Validation Statistics (V1–V5)

We report validation metrics across training epochs for each prototype. Unless stated otherwise, the “best” checkpoint corresponds to the highest validation F1 observed during the run (as saved by our code).

V1: Dense-RAG Hybrid + Top- k Sparse Neural Memory.

- Best Val F1: **0.7083** (Epoch 01; also matched at Epoch 02)
- Best Val Balanced Accuracy: **0.6667** (Epoch 01/02)
- Trend: training loss decreases monotonically, while validation performance fluctuates and degrades after early epochs, indicating instability/overfitting under the current representation + retrieval mismatch.

V2: Hash-Slot Memory + TF-IDF Retrieval Hybrid.

- Best Val F1: **0.8682** (Epoch 01–05)
- Best Val Balanced Accuracy: **0.8333** (Epoch 01–05)
- Trend: very strong early performance, followed by sharp degradations after Epoch 06 (lowest around Epoch 13), suggesting that later training steps introduce drift in the memory traces or increase reliance on spurious lexical shortcuts.

V3: Pure BDH (No Retrieval) with Time/Location Causal Memory.

- Best Val F1: **0.4808** (flat across Epoch 01–15)
- Balanced accuracy was not printed in this run; F1 flatness suggests near-chance behavior and a failure to couple ingestion dynamics to the supervised label structure.
- Trend: ingestion loss decreases substantially (memory learns to reconstruct), but supervised consistency does not improve, consistent with an *objective mismatch* (reconstruction success does not imply discriminative narrative state).

V4: Graph-RAG Hybrid (Knowledge Graph Retrieval) + Top- k Memory.

- Best Val F1: **0.7935** (Epoch 01–03)
- Best Val Balanced Accuracy: **0.7500** (Epoch 01–03)
- Trend: strong early performance, then gradual degradation; graph retrieval helps relative to V1 but is likely sensitive to NER aliasing and spurious co-occurrence neighborhoods.

V5: Hash-Slot CausalMemory with Time/Location Gating (Threshold-Calibrated).

- Best Val F1: **0.9170** (Epoch 05)
- Threshold calibration on training scores yields: $\tau^* = 0.3333$
- Trend: validation improves steadily through Epoch 05, unlike earlier prototypes that peaked immediately. The calibrated threshold removes an additional source of brittleness observed in fixed- τ variants.

13.2 Compact Summary Table (Best Checkpoints)

Proto	Retrieval	Memory dressing	Ad-	Best F1	Val	Best Val Bal-Acc	Observed Training Behavior
V1	Dense (SBERT)	Top- k (content)		0.7083		0.6667	Early peak; later drift/overfit; retriever-classifier mismatch
V2	TF-IDF	Hash-slot (entity)		0.8682		0.8333	Very strong early; later epochs degrade sharply (trace drift / shortcut learning)
V3	None	Top- k (content) + T/L		0.4808		–	Reconstruction improves but supervision flat (objective mismatch)
V4	Graph-RAG	Top- k (content)		0.7935		0.7500	Early peak then degradation; NER/co-occurrence confounds
V5	None	Hash-slot (entity) + T/L		0.9170		–	Most stable improvement; adds calibration ($\tau^* = 0.3333$)

13.3 Team Iteration Narrative: Why We Started at V1 and Settled at V5

From a team perspective, our iteration path was a sequence of falsifiable hypotheses about long-context narrative reasoning under severe compute and modeling constraints.

Why V1 first (Dense-RAG + Top- k). We began with V1 because it is the most standard “long-context” recipe: a *persistent global memory* for compression plus a *retriever* for selective expansion. The intended failure mode it addresses is: “relevant evidence exists in the novels, but is too far back to fit into a local context window.” Dense retrieval (SBERT) is designed to recover semantically related evidence even under paraphrase.

What V1 taught us. The validation curve showed an early peak and then degradation despite lower training loss. In our setting, two structural issues likely dominated:

- **Embedding-space mismatch:** SBERT retrieval optimizes $\cos(\phi(q), \phi(c))$ in a different space than the BDH encoder/memory, so retrieved chunks are not guaranteed to be “useful” under the classifier’s geometry.
- **Addressing misalignment:** Top- k indices are driven by claim content tokens. This gives a working memory keyed by *what the claim looks like*, not by *which entity the claim is about*. That is often suboptimal for entity-centric consistency detection.

Why V2 next (TF-IDF + Hash-slot). V2 was motivated by a more data-aligned hypothesis: if claims and evidence share key entity strings and explicit lexical cues, *lexical* retrieval can outperform semantic retrieval while being cheaper and more stable. Simultaneously, hash-slot addressing provides deterministic entity indexing at probe time via $e(\text{char})$, which is closer to the evaluation format.

What V2 taught us. V2 achieved strong early metrics (Val F1 0.8682, BalAcc 0.8333) but degraded later. This is consistent with *memory drift*: as the model continues ingesting and updating, hash-slot representations can become contaminated by collisions and repeated writes, reducing discriminability. This suggested we needed more explicit stabilization (time/location conditioning and better decision calibration).

Why V3 (remove retrieval entirely). V3 was a deliberate ablation: can a sufficiently stabilized memory (Top- k + time/location) replace retrieval? The result was flat at Val F1 0.4808, indicating that our ingestion objective was learning reconstruction dynamics but not creating entity-discriminative state. Practically, it revealed that without retrieval (and without entity-addressed probing), the system cannot reliably surface the right evidence.

Why V4 (Graph-RAG). V4 was an attempt to keep retrieval but make it *entity-structured* rather than purely semantic: narrative consistency is often driven by relationships among entities. Graph-RAG improved over V1 (best Val F1 0.7935), supporting the idea that entity-centric priors are helpful, but it still exhibited degradation and is vulnerable to NER aliasing/spurious co-occurrence.

Why V5 is where we settled. V5 combined the key lessons:

- **Entity-addressed probing (hash-slot):** the queried memory trace is directly tied to `char` via $e(\text{char})$.
- **Stabilized updates (time/location gating):** updates are conditioned by coordinates to reduce overwrite drift and make repeated terms less destructive.
- **Calibration is not optional:** selecting τ^* by sweeping training scores removes brittleness introduced by fixed thresholds and shifts in score distributions across runs.

This combination produced the most consistently improving validation trajectory and our best observed validation F1 (0.9170).

13.4 Why We Still Suspect Failures on This Task (Even in V5)

Despite strong validation, several model-intrinsic reasons remain for failure, especially under distribution shift:

(1) Bag-of-words semantics fundamentally limit contradiction detection. All prototypes use mean pooling:

$$\mathbf{b} = \frac{1}{L} \sum_{i=1}^L \mathbf{x}_i,$$

which discards order and scope. Statements differing only by negation, temporal qualifiers, or role reversal can map to nearby embeddings and yield incorrect cosine decisions.

(2) Hash collisions create persistent interference. For hash-slot models,

$$e(x) = e(x') \quad \text{for } x \neq x',$$

forces unrelated entities/tokens to share the same row. This yields cross-entity contamination and irreducible noise in $\mathbf{M}_{e(\cdot)}$.

(3) Ingestion/supervision coupling remains imperfect. In V5, ingestion updates word slots $e(w)$ while supervision probes entity slots $e(\text{char})$. This can stabilize the memory mechanism but does not guarantee that entity-specific narrative facts are written to the entity slot unless the training distribution implicitly induces it. This is a structural limitation, not a hyperparameter issue.

(4) Threshold calibration can overfit if the score distribution shifts. Even though V5 calibrates τ^* , the calibration is performed on training scores. If test-time score distribution shifts (different entity frequency, different writing style, different claim length), τ^* can be miscalibrated.

(5) Multi-entity claims are under-modeled. Claims like “X betrayed Y” are scored against a single entity trace. Without explicit relation modeling, the system can misattribute roles and treat relational contradictions as consistent.

(6) Truncation and per-token updates bias what is learned. All runs truncate chapters/words for runtime. This biases the memory toward high-frequency narration patterns rather than rare, causally decisive facts.

13.5 Prototype-Specific Failure Conditions (When Each Model Breaks)

This subsection enumerates concrete, prototype-specific failure regimes. These are not generic limitations; each item is tied to a mechanism unique to the design choices of the corresponding version.

13.5.1 V1 Failure Conditions: Dense-RAG (SBERT) + Top- k Memory

V1 fails most reliably under the following conditions:

- **Retriever–classifier embedding mismatch.** Retrieval is driven by SBERT similarity

$$\cos(\phi(q), \phi(c)),$$

while classification operates in the BDH embedding geometry. If the retrieved chunks are semantically close under $\phi(\cdot)$ but not informative under BDH pooling, the local trace becomes an off-manifold distractor and can reduce the final score

$$s = \alpha \cos(\mathbf{r}_g, \mathbf{b}) + (1 - \alpha) \cos(\mathbf{r}_\ell, \mathbf{b}).$$

- **High-lexical-overlap contradictions.** When contradictory claims share most tokens with the consistent version (negation flips, antonyms), both the query and retrieved chunks become lexically similar, inflating $\cos(\mathbf{r}_\ell, \mathbf{b})$ and yielding false “consistent”.
- **Top- k hard selection brittleness.** The hard operator

$$\mathcal{I} = \text{TopK}(\mathbf{L}, k)$$

causes discontinuous addressing. Small perturbations in embeddings can shift $\mathcal{I}$, producing unstable memory traces and epoch-to-epoch volatility in validation.

- **Dense retrieval recall collapse for rare/alias entities.** If entity names in `train.csv` differ from novel mentions (aliases, honorifics, partial names), SBERT can retrieve irrelevant high-level chunks; the local trace becomes noise.
- **Compute truncation amplifies retrieval dominance.** Because ingestion reads only a subset of the novels, the global memory trace $\mathbf{r}_g$ is weak; the model over-relies on $\mathbf{r}_\ell$, making performance hostage to retrieval quality.

13.5.2 V2 Failure Conditions: Hash-Slot Memory + TF-IDF Retrieval

V2 fails most reliably under the following conditions:

- **Paraphrase and synonymy (TF-IDF blindness).** TF-IDF retrieval maximizes lexical overlap

$$\psi(q)^\top \psi(c),$$

so semantically equivalent evidence with different wording is missed, yielding retrieval dropout and forcing the model to guess from a weak global trace.

- **Keyword confounds (lexical shortcut learning).** If certain words correlate with labels (e.g., “dead”, “alive”, “married”), TF-IDF preferentially retrieves chunks containing them even when irrelevant, producing spurious evidence that inflates $\cos(\mathbf{r}_\ell, \mathbf{b})$.
- **Hash collision contamination (persistent and unrecoverable).** When

$$\exists x \neq x' : e(x) = e(x'),$$

updates intended for x alter the same slot used for x' , contaminating $\mathbf{M}_{e(\cdot)}$ across both ingestion and supervision.

- **Late-epoch memory drift.** With repeated updates to the same hash slot, the recurrence

$$\mathbf{m}' = f \odot \mathbf{m} + i \odot \mathbf{z}$$

can shift $\mathbf{m}$ toward recent high-frequency patterns, degrading earlier alignment; this matches the observed “strong early, worse later” curve.

- **Entity not explicitly present in claim.** When **content** relies on pronouns/titles rather than explicit names, TF-IDF retrieval underperforms (query terms missing), and the entity slot may not contain corresponding evidence.

13.5.3 V3 Failure Conditions: Pure BDH + Time/Location Causal Memory (No Retrieval)

V3 fails most reliably under the following conditions:

- **No retrieval fallback for missing evidence.** If the relevant facts are not written into the top- k addressed subset of memory, inference cannot recover. There is no external evidence expansion.
- **Objective mismatch: reconstruction does not imply discriminative state.** Ingestion optimizes

$$\mathcal{L}_{\text{recon}} = \|\text{Rec}(\mathbf{m}') - \mathbf{f}\|_2^2,$$

which can be minimized even if the resulting memory does not separate consistent vs. contradictory claims. This explains stable reconstruction improvements with flat F1.

- **Entity grounding failure (content-addressing not entity-addressing).** Probing uses indices derived from claim tokens (Top- k on content), not from entity identifiers. The system thus builds a content-conditioned trace rather than an entity-conditioned trace, misaligning with the dataset’s **char**-centric supervision.
- **Temporal binding is underused at inference.** Time/location affects updates

$$\tilde{\mathbf{f}} = \mathbf{f} \odot (\mathbf{t} + \ell),$$

but inference does not condition on the corresponding coordinates for the claim. Thus the model cannot reliably exploit “when/where” distinctions at decision time.

- **Truncation sensitivity.** Because ingestion is truncated, the learned memory captures partial narrative statistics and fails on claims requiring rare long-range facts.

13.5.4 V4 Failure Conditions: Graph-RAG + Top- k Memory

V4 fails most reliably under the following conditions:

- **NER aliasing and entity fragmentation.** Graph construction depends on entity extraction $\mathcal{E}(c)$; aliases (“Dantès” vs “Edmond”, titles, spelling variants) split nodes, weakening edges and causing retrieval dropout.

- **Spurious co-occurrence neighborhoods.** Edges encode frequency

$$w(u, v) = \sum_m \mathbb{I}[u, v \in \mathcal{E}(c_m)],$$

but co-occurrence reflects narrative proximity, not factual relevance. High-degree hubs introduce confounds: retrieval brings socially adjacent but irrelevant chunks.

- **Query entity extraction failure.** If query entities $\mathcal{Q}$ are missed, the fallback to capitalized words can retrieve systematically wrong neighborhoods (e.g., common capitalized nouns), producing noisy evidence.
- **One-hop limitation.** Using only 1-hop expansion

$$\mathcal{N}(\mathcal{Q}) = \mathcal{Q} \cup \bigcup_{q \in \mathcal{Q}} \text{TopNeighbors}(q)$$

can miss relevant evidence that is 2+ hops away (indirect relations), yielding recall failures for relational claims.

- **Top- k address instability mirrors V1.** Graph retrieval can help evidence targeting, but the same hard Top- k addressing can still cause unstable memory traces and sensitivity to small embedding shifts.

13.5.5 V5 Failure Conditions: Hash-Slot CausalMemory + Time/Location Gating + Calibration

V5 fails most reliably under the following conditions:

- **Entity/ingestion mismatch remains structural.** Ingestion updates $e(w)$ while inference probes $e(\text{char})$. Therefore, even with stable dynamics, entity slots may be under-populated with relevant narrative facts unless the training signal explicitly pushes them there.
- **Calibration shift under distribution change.** Threshold selection

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \text{BalAcc}(\mathbb{I}[s \geq \tau], y)$$

can overfit the training score distribution. If test scores shift (different length/style/entity mix), τ^* becomes miscalibrated.

- **Hash collisions persist and are amplified by persistence.** Collisions in $e(\cdot)$ are irreversible: repeated gated updates produce blended traces that degrade cosine separability.
- **Negation/role-reversal failures (BoW pooling).** Because claim embedding is mean pooled, contradictions depending on syntax can remain near the consistent embedding in cosine space, causing false positives.
- **Multi-entity relational statements.** A single entity trace $\text{Rec}(\mathbf{M}_{e(\text{char})})$ cannot explicitly represent relations (X did Y to Z) without additional relational slots; role confusion remains likely.

14 Primary Metrics, Calibration Logic, and Practical Target Ranges

14.1 Which metric matters most

Across the prototypes, two metrics recur: weighted F1 and balanced accuracy. For checkpoint selection during training, weighted F1 is often the most visible metric because it reflects overall classification quality under a swept threshold. For deployment, however, the strongest metric is **balanced accuracy**, because V5 explicitly calibrates the operating threshold to maximize it.

This distinction matters. Weighted F1 is useful for comparing checkpoints; balanced accuracy is the better operating metric once threshold choice becomes part of the model.

14.2 Why balanced accuracy is the primary deployment metric

Balanced accuracy gives equal importance to both classes. That matters because a thresholded cosine system can look good under ordinary accuracy while favoring one class or exploiting class imbalance. Once the decision boundary τ^* is chosen by sweeping balanced accuracy, the project has effectively declared that metric to be the primary deployment objective.

14.3 Why weighted F1 still matters

Weighted F1 remains valuable because the prototypes do not output calibrated probabilities; they output a similarity score that must be thresholded. Weighted F1 summarizes the precision–recall trade-off of that thresholded decision and is a good model-selection diagnostic during training.

14.4 Practical target ranges

For this task family, the following bands are reasonable engineering guides rather than universal laws:

- **Balanced accuracy:** above 0.85 is strong, 0.80–0.85 is good, 0.70–0.80 is workable but brittle, and near 0.50 is effectively random.
- **Weighted F1:** above 0.85 is strong, 0.78–0.85 is respectable, and below roughly 0.75 usually signals unstable contradiction behavior or poor calibration.

14.5 What a good model should look like beyond one scalar

A genuinely strong BDH-style model should not only score well at its best threshold; it should also show:

- stability across nearby thresholds,
- limited gap between F1 and balanced accuracy,
- resistance to lexical-overlap traps,
- graceful degradation under paraphrase and negation.

15 Hyperparameter Roles and Why These Values Make Sense

15.1 Shared representation scale

$D = 256$. This is the latent width shared by the embedding table, memory state, reconstruction head, and similarity space. It is large enough to encode multiple latent factors, but small enough to keep the memory matrix and update networks lightweight. For a prototype exploring memory behavior rather than maximizing NLP semantics, 256 is a reasonable middle ground.

$N = 32768$. This is the memory size in most variants. It is large enough to reduce the probability of catastrophic collisions relative to small memories, but still tractable on a Kaggle-style GPU. A much smaller N would push unrelated entities or words into the same slot too often; a much larger one would increase memory cost without guaranteeing better supervision.

$V = 50257$. The pseudo-vocabulary size mirrors a familiar tokenizer scale and keeps hash collisions somewhat manageable. Since the tokenizer is not learned in the usual subword sense, this value mainly controls collision pressure rather than semantic granularity.

$H = 4$. The head count in the encoder-side routing tensor gives modest multi-view addressing without exploding the routing complexity. It is large enough to support some diversity in active indices, but still simple enough to keep the sparse-addressing logic understandable.

$\text{TOP_K} = 8$. In Top- k variants, eight active slots per routed token sequence is a moderate sparsity level. Too small a value would underrepresent the fact; too large a value would destroy the sparsity advantage and increase routing instability.

15.2 Retrieval hyperparameters

$K_{\text{retrieval}} = 3$. Retrieving three chunks is a conservative choice. It gives the local stream enough evidence diversity to help the classifier, while keeping noise bounded. Larger retrieval sets often improve recall but quickly increase confounding when retrieval quality is imperfect.

Chunk size 120 (Dense-RAG), 150 (TF-IDF), 200 (Graph-RAG). The chunk size is tailored to the retriever family. Dense retrieval benefits from shorter chunks because embedding quality usually drops when passages become too broad. TF-IDF tolerates slightly larger chunks because lexical overlap still remains meaningful. GraphRAG uses larger chunks because co-occurrence structure benefits from broader local context.

Dense retriever model all-MiniLM-L6-v2. This is a lightweight, well-known semantic retriever that fits the project’s preference for rapid iteration. It is not the strongest possible dense retriever, but it is a reasonable baseline when the goal is to compare retrieval families rather than squeeze out state-of-the-art semantic recall.

TF-IDF max_features = 50000. This keeps the sparse lexical space expressive enough for entity-rich evidence while remaining computationally manageable.

15.3 Optimization and training schedule

EPOCHS = 15 in the multi-prototype runs. Fifteen epochs is enough to observe whether a design stabilizes, peaks early, or drifts. That makes sense for a comparative research notebook, even if production tuning would require a longer and more systematically validated schedule.

MAX_LEN = 100. This acts as a runtime bound for the simplified pipelines. It is not a semantic optimum; it is a tractability choice that limits the compute cost of prototyping.

Chapter/word truncation such as 10 chapters $\times$ 150 words, 15 chapters $\times$ 100 words, or 20 chapters $\times$ 150 words. These are not conceptually ideal settings. They are compute-budget approximations used to make long-context experiments feasible. Their role is important because they directly affect which facts the memory ever sees.

15.4 Loss weights

$W_{\text{RECON}} = 1.0$. Reconstruction is the anchor objective for ingestion. Setting it to 1.0 makes it the reference scale around which the auxiliary regularizers are tuned.

$W_{\text{GATING}} = 0.1$. This penalizes indiscriminate writes without overpowering the memory’s ability to adapt. If it were much larger, the model could become too conservative and fail to store new evidence at all.

$W_{\text{DECAY}} = 0.01$. This gently encourages forget gates to stay near retention unless the model has a strong reason to erase. The small value is sensible because too much pressure on the forget term would freeze the memory.

$W_{\text{CONSISTENCY}} = 2.0$. The supervised objective is deliberately upweighted relative to the ingest regularizers so that, once memory mechanics are stable enough, label separation dominates model selection.

15.5 Threshold sweep choices

Threshold grids such as 50 points over $[-0.5, 0.5]$ or $[-1, 1]$. These sweeps are coarse but practical. Since cosine similarity is bounded and usually concentrated in a relatively narrow range, a 50-point sweep is sufficient for notebook-scale calibration. In a production pipeline, the sweep would ideally be denser and validated across folds.

$\tau^* = 0.3333$ in V5. The calibrated threshold reported in the draft is important because it proves that zero-threshold cosine is not the optimal operating point once time/location-conditioned entity memory is used. That is a deployment lesson, not just a numeric artifact.

16 Senior-Level Interview Questions and Model Answers

16.1 Why external memory instead of just using a larger language model?

Because the project is explicitly about persistent state. A larger language model may be more semantically expressive, but it hides the persistence mechanism inside one contextual representation. BDH makes storage, forgetting, collisions, and calibration observable and tunable.

16.2 Why mean pooling when you know it discards syntax?

Because the goal of the project is to isolate memory and retrieval effects under a deliberately simple encoder. Mean pooling is not the semantically strongest choice, but it makes failure modes transparent and keeps the prototype lightweight enough to compare multiple memory designs.

16.3 Why use a reconstruction head at all?

Without a reconstruction objective, the memory can drift into an arbitrary latent state that is no longer interpretable or comparable. Reconstruction regularizes the memory so that writing remains recoverable.

16.4 Why are time and location embeddings useful?

They reduce destructive interference from repeated words across long narratives. The same token should not have an identical effect in every chapter. Position-conditioned updates give the memory a minimal notion of *where* and *when* evidence occurred.

16.5 Why did V5 become the preferred version?

Because it combines the most important lessons from earlier prototypes: direct entity-addressed probing, stabilized time/location-gated updates, and threshold calibration. That combination was the most operationally stable design even if it did not solve every semantic limitation.

16.6 Why can V2 or V5 outperform semantically richer models on some runs?

Because the dataset format is explicitly entity-conditioned. A deterministic entity-addressed slot can be more aligned to the supervision signal than a semantically richer but less targeted routing mechanism.

16.7 What is the single biggest unresolved weakness?

The bag-of-words claim encoder. It discards order, negation scope, and relational structure, so high lexical-overlap contradictions remain fundamentally difficult.

16.8 What would be the next senior-level improvement?

Keep the explicit memory idea, but replace mean pooling with a contextual encoder, move from scalar gates to feature-wise gates, and add relation-aware or multi-entity probing so the memory aligns more naturally with narrative statements involving more than one actor.

17 Conclusion

BDH On Top is best understood as a structured-memory research project rather than a conventional text classifier. Its strongest idea is not any single prototype, but the consistent attempt to turn long-context narrative reasoning into an explicit state-tracking problem. The most successful variant, V5, shows that entity-addressed probing, time/location-conditioned updates, and calibrated thresholding can produce a meaningfully more stable decision system than either pure retrieval or unstable sparse routing alone.

At the same time, the project also makes its own limitations visible: bag-of-words semantics, collisions, retrieval mismatch, and calibration sensitivity remain real failure modes. That transparency is a strength. It means the next iteration can be improved systematically rather than guessed at.